UART 통신 기반
센서 2종 및 시계,스톱워치
통합 제어 시스템



5조 : 조수영, 박경훈, 오창민, 박재균

날짜: 2026.11.02

Contents

01 개요

02 블럭 다이어그램

03 Related Technical

04 시뮬레이션 시나리오

05 시뮬레이션 결과

06 Implementation

07 동작 영상

08 결론 및 고찰

팀 소개, 역할 분담

조수영

- ascii sender, decoder 설계 검증
- 스위치에 따른 fnd 표시모드 검증

박경훈

- Control Unit 설계
- Top Module 검증 testbench 작성

오창민

- Top Integration
- FPGA Implementation & Verification

박재균

- RS04 설계 및 검증
- DHT11 설계 및 검증

프로젝트 개요

설계 목적

UART를 통한 cmd 명령 · 버튼 → 4개 모드 제어 (시계 · 스톱워치 · 초음파 · 온습도) → FND · UART 출력



하드웨어 · 소프트웨어

- 보드 : Basys 3 (Artix-7, 100 MHz)
- 센서 : HC-SR04 · DHT11
- 입력 : 버튼 4개 · 스위치 4개
- 툴 : Verilog · Vivado · ComportMaster

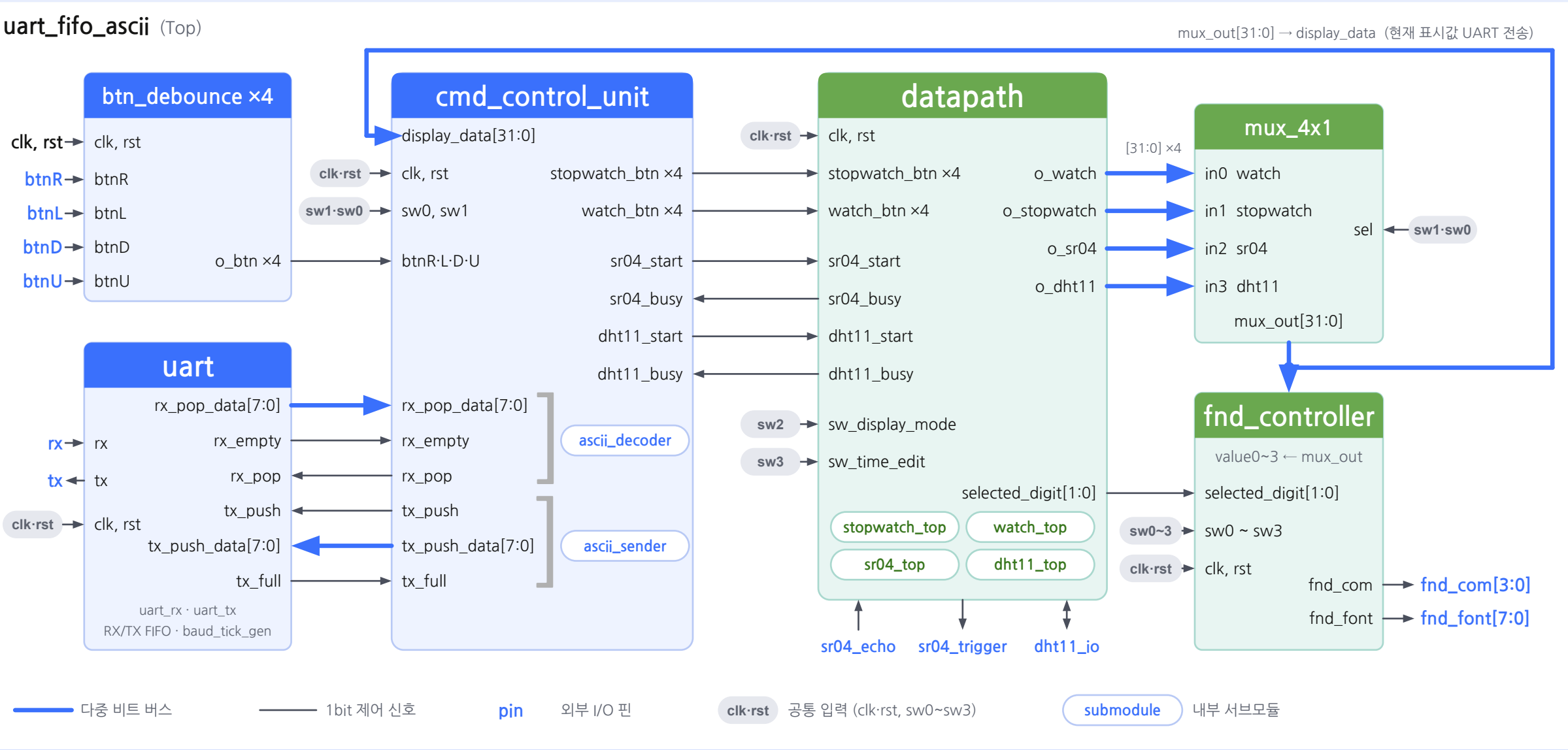
설계 사양

동작 모드	sw1·sw0 = 00 시계 · 01 스톱워치 · 10 SR04 · 11 DHT11
UART 명령	R·L·D·U = 버튼 동작 · S = 현재 값 전송
UART 응답	DONE 성공 · FAIL 측정 중 · NONE 미지원
통신 형식	9600 bps 8N1 · 16배 오버샘플링 · FIFO 4 × 8bit
FND 표시	sw0~2 표시 전환 · sw3 시계 편집

결과 요약

- **버튼**(R·L·U·D)과 **UART 명령**(R·L·U·D) 모두로 **시계 · 스톱워치 · 센서 2종 제어** 가능
- '**S**' 명령 시 **현재 모드의 값**(시각 · 거리 · 온습도)을 tx로 출력
- 명령 처리 결과 회신 : 성공 → DONE, 실패 → FAIL (예: 센서 측정 중 측정 명령), 미지원 명령 → NONE
- SR04는 58 µs = 1 cm로 거리 계산, DHT11은 단계별 100 µs 타임아웃 및 체크섬 일치 시에만 값 갱신

블럭 다이어그램

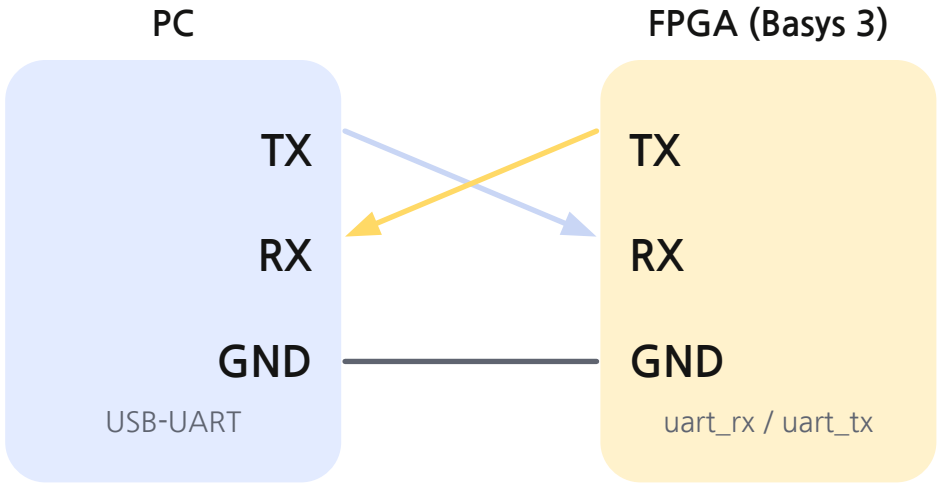

uart_fifo_ascii (Top)
mux_out[31:0] → display_data (현재 표시값 UART 전송)
btn_debounce ×4
clk, rst
btnR
btnL
btnD
btnU
clk, rst
btnR
btnL
btnD
btnU
o_btn ×4
cmd_control_unit
display_data[31:0]
clk·rst
clk, rst
sw1·sw0
sw0, sw1
btnR·L·D·U
stopwatch_btn ×4
watch_btn ×4
sr04_start
sr04_busy
dht11_start
dht11_busy
rx_pop_data[7:0]
rx_empty
rx_pop
tx_push
tx_push_data[7:0]
tx_full
ascii_decoder
ascii_sender
datapath
clk·rst
clk, rst
stopwatch_btn ×4
watch_btn ×4
sr04_start
sr04_busy
dht11_start
dht11_busy
sw2
sw_display_mode
sw3
sw_time_edit
o_watch
o_stopwatch
o_sr04
o_dht11
selected_digit[1:0]
stopwatch_top
watch_top
sr04_top
dht11_top
sr04_echo
sr04_trigger
dht11_io
[31:0] ×4
mux_4x1
in0 watch
in1 stopwatch
in2 sr04
in3 dht11
sel
sw1·sw0
mux_out[31:0]
uart
rx_pop_data[7:0]
rx
rx_empty
tx
rx_pop
tx_push
clk·rst
clk, rst
tx_push_data[7:0]
tx_full
uart_rx · uart_tx
RX/TX FIFO · baud_tick_gen
fnd_controller
value0~3 ← mux_out
selected_digit[1:0]
sw0~3
sw0 ~ sw3
clk·rst
clk, rst
fnd_com
fnd_font
fnd_com[3:0]
fnd_font[7:0]
다중 비트 버스
1bit 제어 신호
pin
외부 I/O 핀
clk·rst
공통 입력 (clk·rst, sw0~sw3)
submodule
내부 서브모듈

Related Technical

UART

Universal Asynchronous Receiver / Transmitter — 병렬 ↔ 직렬 변환으로 데이터를 주고받는 비동기 직렬 통신 인터페이스





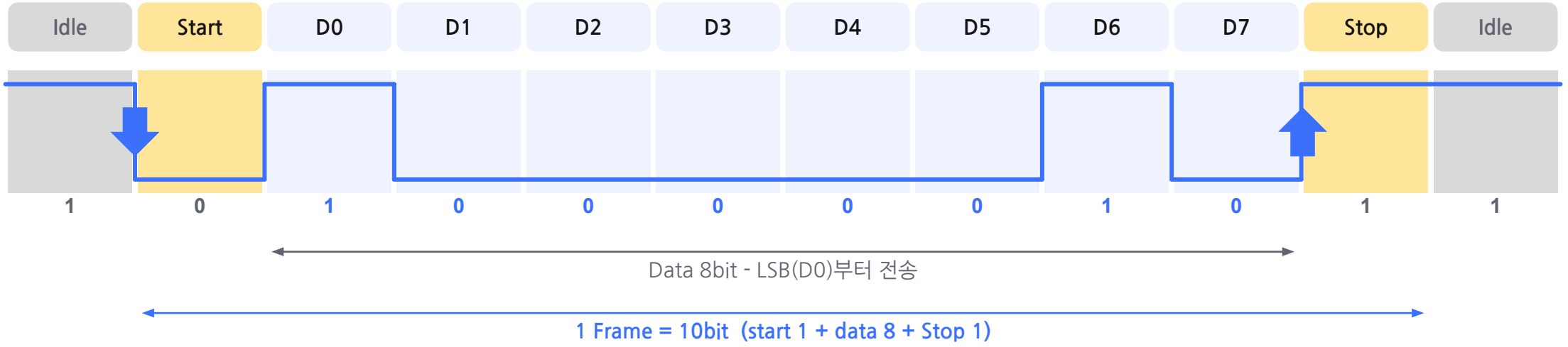
TX는 상대 RX로 교차 연결, GND는 공통 연결

PC → FPGA (파랑) · FPGA → PC (주황)

1. **송신 TX · 수신 RX 분리 → Full-duplex**
   송신선과 수신선이 따로 있어, 보내는 **동시**에 받을 수 있음
2. **비동기 방식**
   보레이트(baud time)만 미리 약속하고 **각자 자기 타이머**로 비트 시간을 잼. 매 프레임 스타트 비트로 타이밍을 다시 맞춤
3. **GND 공통 연결**
   전압은 기준점과의 차이 → 두 장치의 0V 기준을 맞추고 전류가 돌아갈 귀환 경로를 제공

Related Technical

UART 데이터 프레임



1 프레임 구성

Idle HIGH(1) — 대기 상태
Start 1→0
Data 5~9bit, 보통 8bit
Parity <선택> 1의 개수로 오류 검출
Stop HIGH

2 LSB First 전송

[7:0]data = 0100 0001
전송 순서 : 1 0 0 0 0 0 1 0

- 하위 비트(D0)부터 차례로 송신
- 수신 측은 오른쪽 시프트하며 원래 순서로 복원

3 Baud Rate · Baud Time

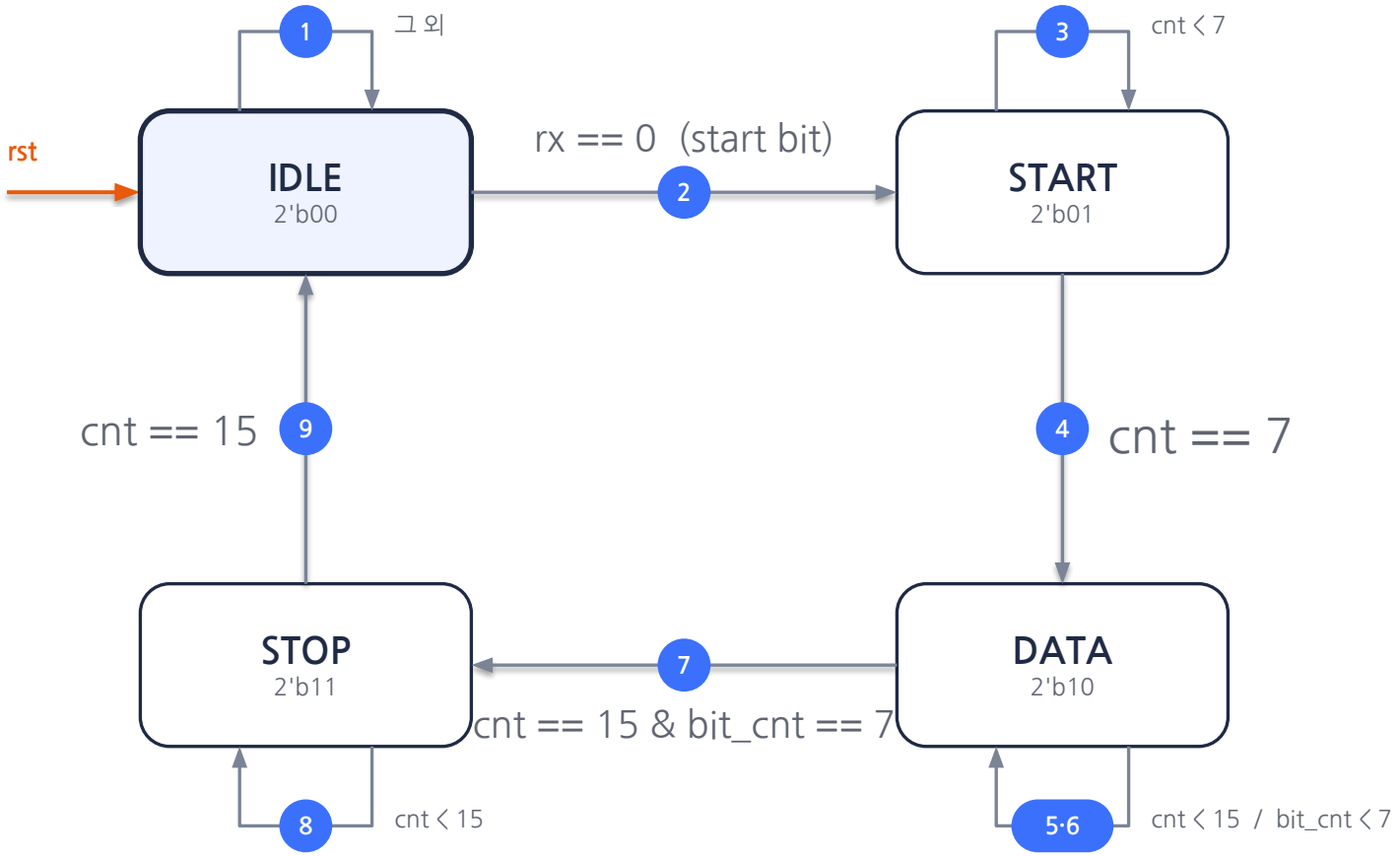
- Baud rate : 초당 비트 수 (9600 bps)

Baud time = 1 / 9600 ≈ 104.16 μs
1 Frame ≈ 1.04 ms → 최대 960 Byte/s

- 송·수신 설정이 다르면 데이터 깨짐

Related Technical

RX FSM



상태 · 인코딩 · 출력 　 리셋 상태 　 n 전이 번호 (우측 전이표 참조)

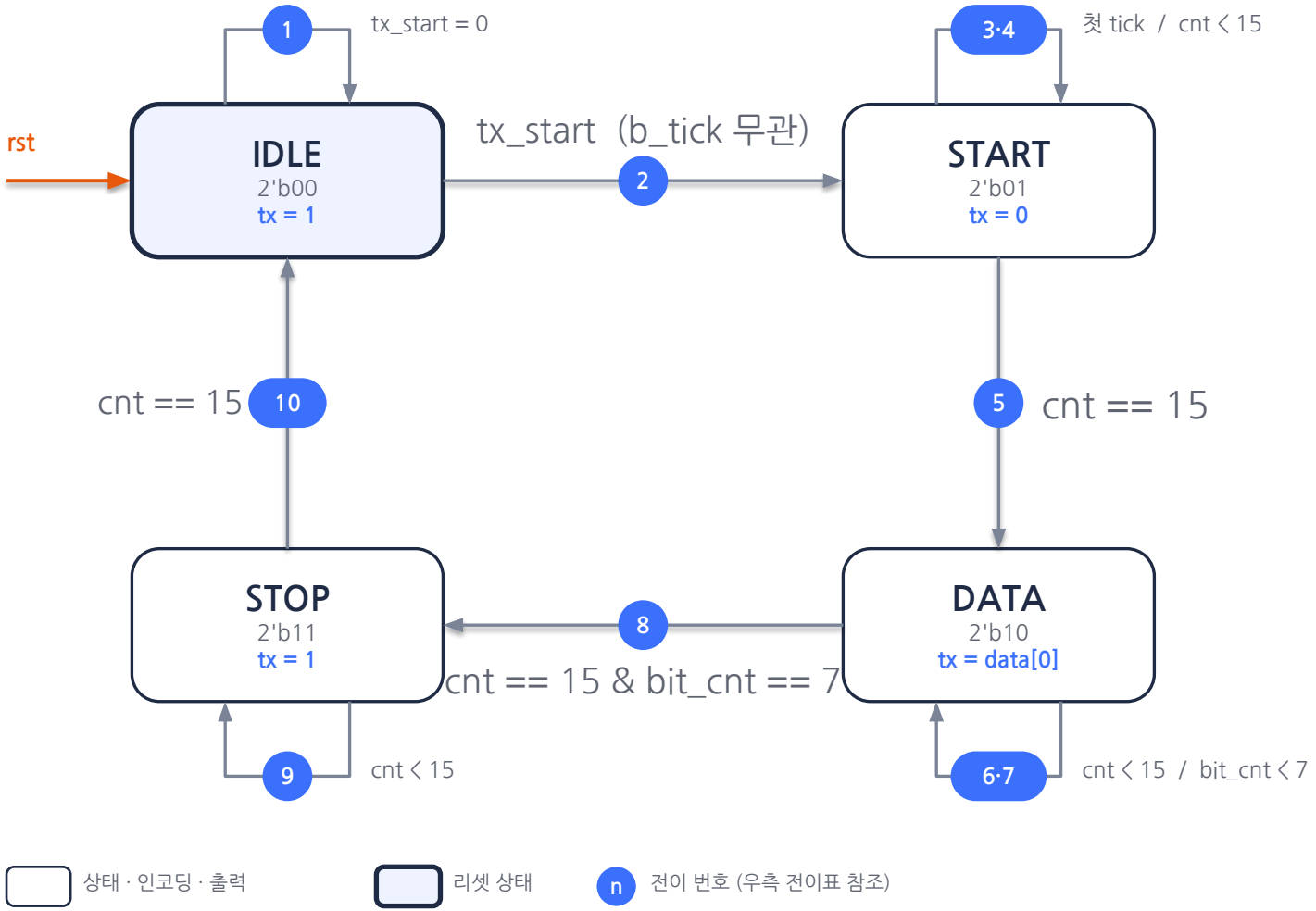
천이 조건표 조건 / 동작 (다음 clk 반영)

No	조건	동작
1	**rx = 1**	대기, rx_done ← 0
2	**rx = 0**	cnt ← 0
3	**cnt < 7**	cnt++ (½ bit 정렬)
4	**cnt = 7**	cnt ← 0, bit_cnt ← 0
5	**cnt < 15**	cnt++
6	**cnt = 15 & bit_cnt < 7**	data ← {rx, data[7:1]} bit_cnt++, cnt ← 0
7	**cnt = 15 & bit_cnt = 7**	data ← {rx, data[7:1]} cnt ← 0
8	**cnt < 15**	cnt++
9	**cnt = 15**	rx_done ← 1 (1 clk 펄스)

※ 모든 조건은 b_tick = 1 일 때만 평가 (b_tick = 16 × baud)

Related Technical

TX FSM



천이 조건표 조건 / 동작 (다음 clk 반영)

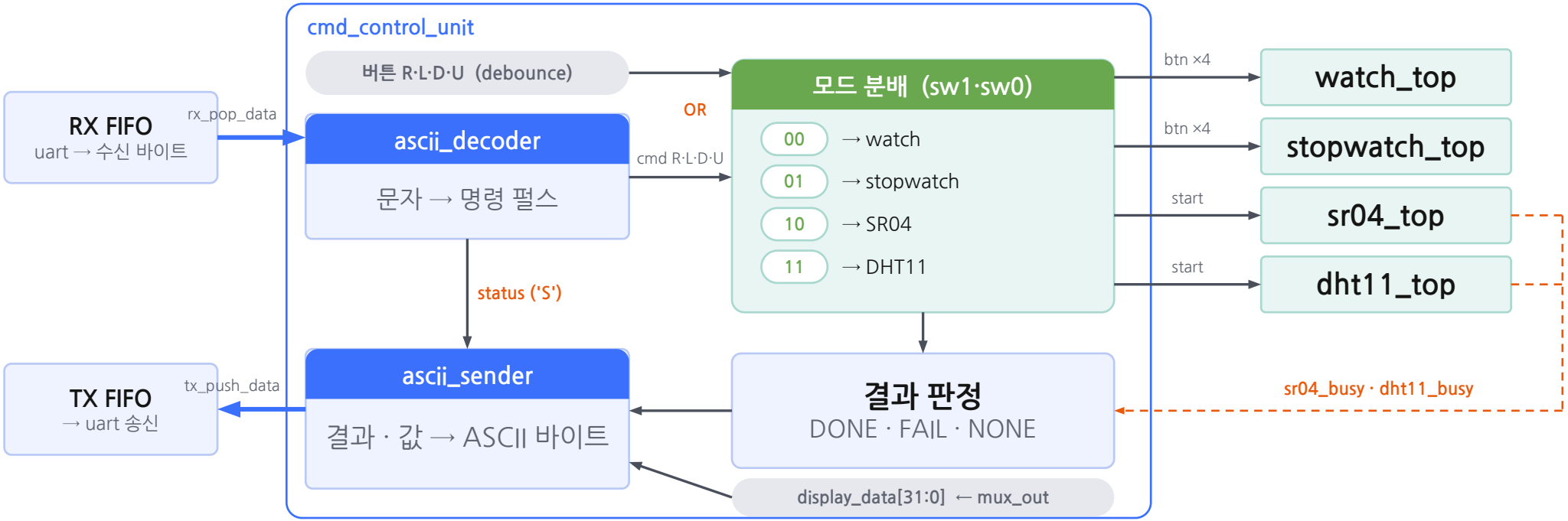
No	조건	동작
1	**tx_start = 0**	대기 (tx = 1 유지)
2	**tx_start**	data ← tx_data, tx_busy ← 1
3	**첫 b_tick (tx = 1)**	tx ← 0, cnt ← 0
4	**cnt < 15**	cnt++
5	**cnt = 15**	tx ← data[0], data >> 1 cnt ← 0, bit_cnt ← 0
6	**cnt < 15**	cnt++
7	**cnt = 15 & bit_cnt < 7**	tx ← data[0], data >> 1 bit_cnt++, cnt ← 0
8	**cnt = 15 & bit_cnt = 7**	tx ← 1, cnt ← 0
9	**cnt < 15**	cnt++
10	**cnt = 15**	tx_busy ← 0

※ 2번만 tx_start로 즉시 전이, 나머지는 b_tick = 1 일 때 평가

Related Technical

cmd_control_unit

목적 PC의 ASCII 명령과 보드 버튼 명령을 하나로 합쳐, 선택된 모듈로 분배하고 처리 결과를 tx로 회신하는 중앙 제어 블록



모드별 명령 처리 ('S'는 모든 모드에서 현재 값 전송)

00 시계
R·L·D·U → watch 버튼
응답 DONE · 기타 문자 → NONE

01 스톱워치
R·L·D·U → stopwatch 버튼
응답 DONE · 기타 문자 → NONE

10 SR04
R → sr04_start
응답 busy면 FAIL · L·D·U·기타 → NONE

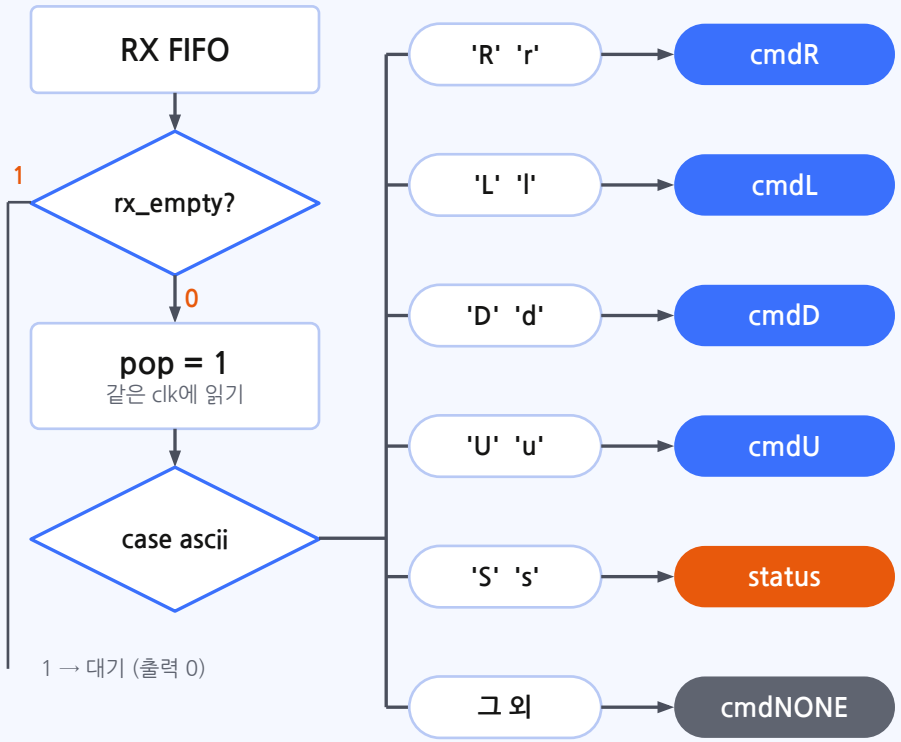
11 DHT11
R → dht11_start
응답 busy면 FAIL · L·D·U·기타 → NONE

Related Technical

ascii_decoder · ascii_sender

ascii_decoder

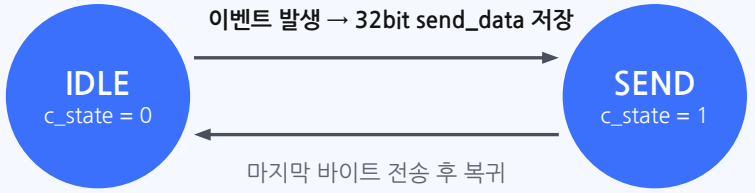
조합 논리 · 대소문자 구분 없음



출력은 1 clk 펄스 → 모드 분배 · ascii_sender로 전달

ascii_sender

1 clk에 1 byte씩 TX FIFO로 push (full이면 대기)



우선순위

1 status (S) → 값 전송
2 success → DONE
3 fail → FAIL
4 cmdNONE → NONE

응답 형식 (마지막 바이트 = 공백)

시계 · 스톱워치	1	2	:	3	4	:	5	6	:	7	8	SP
SR04	1	2	3	c	m	SP	// 앞자리 0일시 생략 ex) 3cm					
DHT11	4	5	%	2	5	*	C	SP				
성공	D	O	N	E	SP							
실패 (busy)	F	A	I	L	SP							
미지원 명령	N	O	N	E	SP							

Related Technical

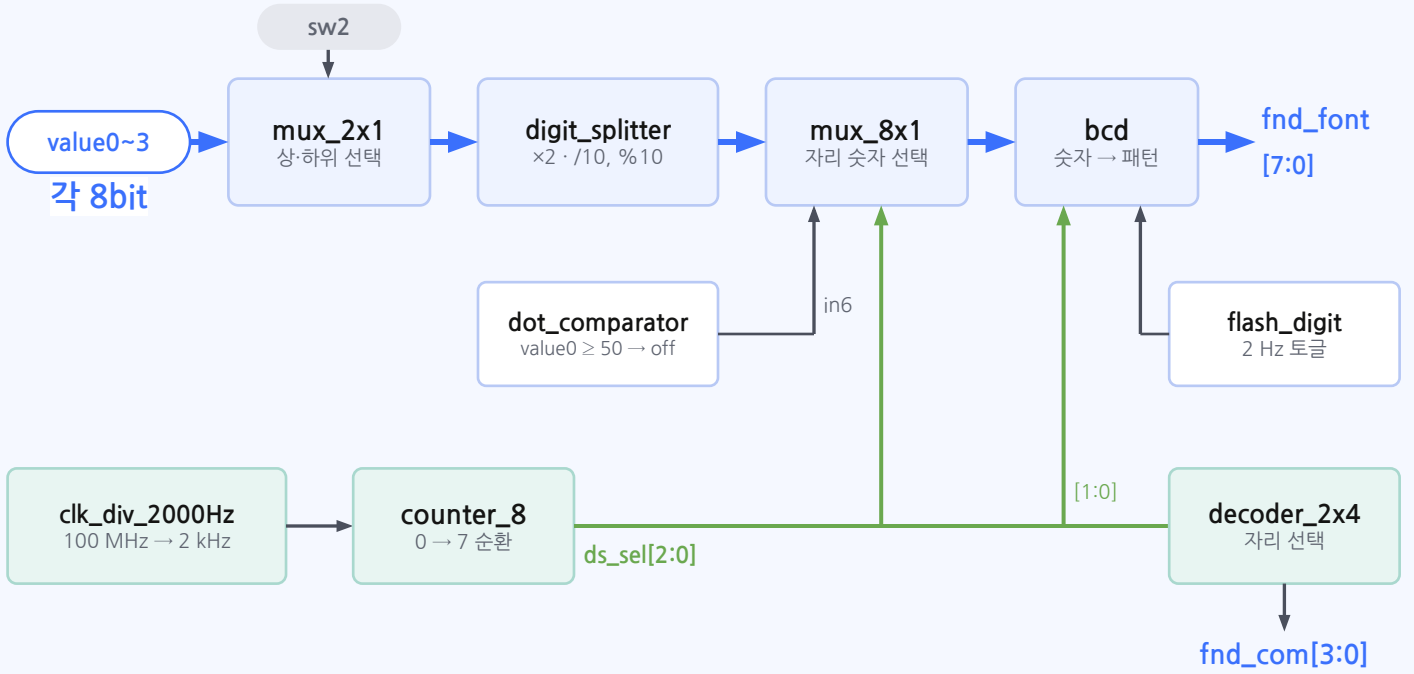
7-Segment · fnd_controller

7-Segment (FND)



- **fnd_font[7:0]** = {dp,g,f,e,d,c,b,a}
- **fnd_com[3:0]** = {com3,com2,com1,com0}
- 0 이면 켜지는 Active Low
- 한 번에 한 자리만 켜짐 -> 빠르게 on/off 잔상으로 4자리 표시

fnd_controller



7segments 표시 전환

(sw0, sw1) 00 :*watch*, 10 :*stopwatch*, 01: *SR04(거리)*, 11: *THD11(온도,습도)*
(sw2) 0: *sec:msec* | 온도, 1: *hour:min* | 습도
(sw3)watch 모드 시 0:수정 모드 x, 1: 수정 모드 o

Related Technical

Ultrasonic sensor (HC-SR04)



정면 기준 왼쪽부터 1·2·3·4번 핀

T: 초음파 송신 / R: 반사파 수신

핀	역할
1 VCC	전원 3.3~5 V*
2 TRIG	FPGA → 센서: 10 µs 시작 펄스
3 ECHO	센서 → FPGA: 왕복 시간 펄스
4 GND	공통 접지

측정 원리 및 출력

- 측정 방식: 초음파 왕복 시간 계수
- 표시 값: 센서와 물체 사이 거리(cm)

거리 = ECHO HIGH 시간 × 음속 ÷ 2

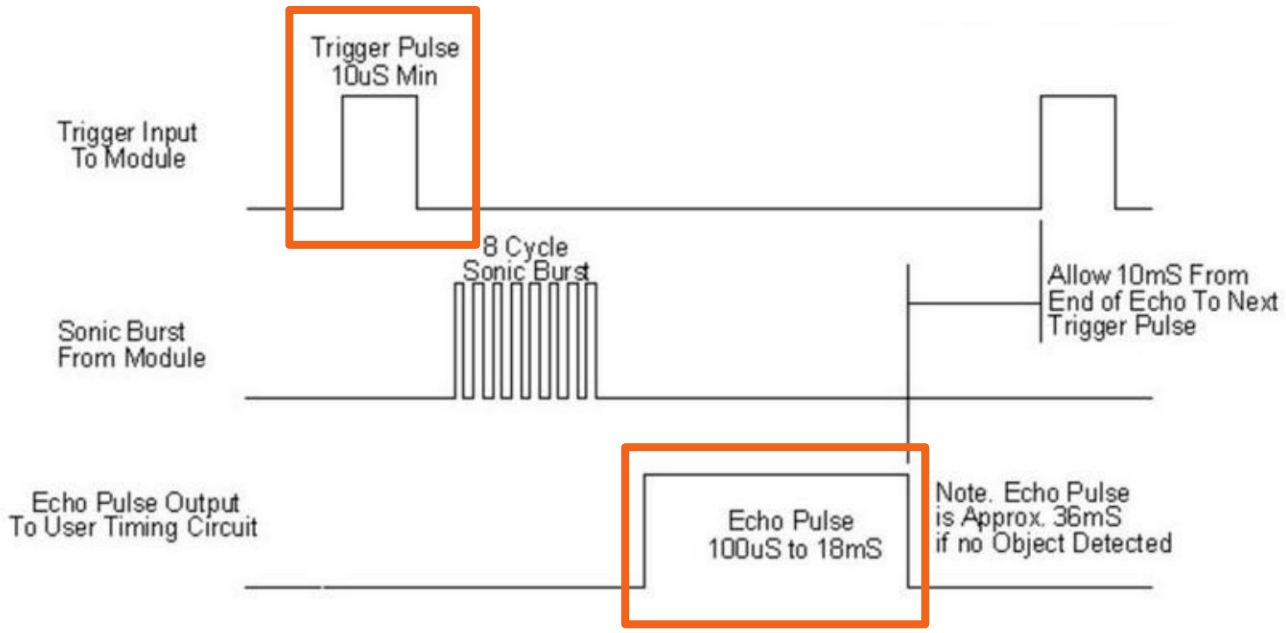
항목	데이터시트 사양
측정 범위	2~400 cm
정확도·오차	±3 mm
초음파 / 유효각	40 kHz / 15°
거리 환산	ECHO HIGH(µs) ÷ 58 ≈ 거리(cm)
측정 주기	60 ms 초과 권장

Related Technical

Ultrasonic sensor (HC-SR04)

ECHO의 HIGH 시간을 세어 거리를 계산

① 10 µs TRIG



② ECHO HIGH 폭

① 측정 시작

TRIG에 10 µs 펄스 입력
센서가 40 kHz 초음파 8주기 송신

② 거리 환산

왕복 시간 ÷ 2로 편도 거리 계산
거리(cm) ≈ ECHO HIGH(µs) ÷ 58

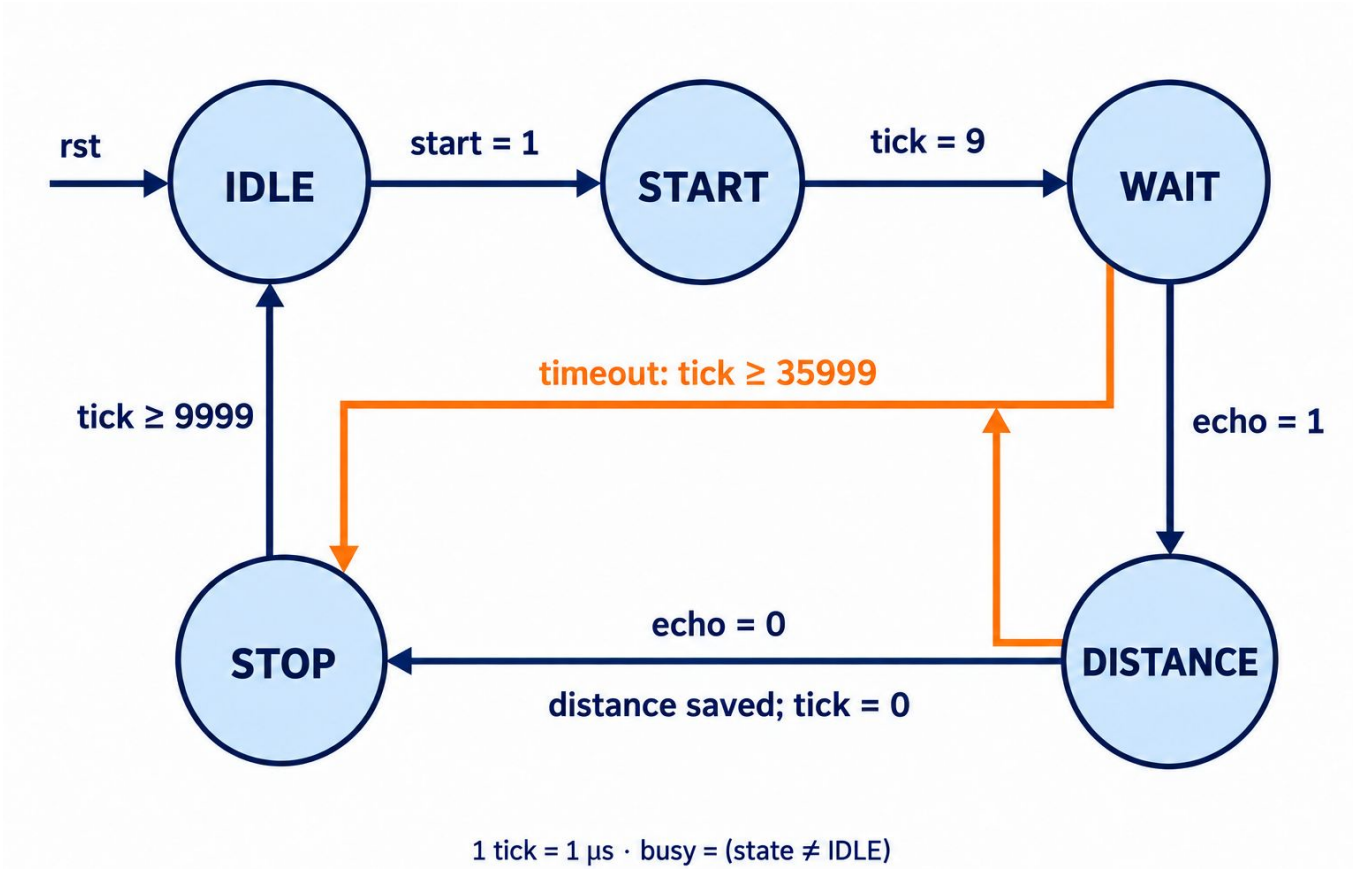
③ 다음 측정 준비

최종 RTL: STOP에서 10 ms 대기
데이터시트: 측정 주기 60 ms 초과 권장

제어 신호: start 입력 / trig 출력 / echo 입력 / busy·done 출력.

Related Technical

Ultrasonic sensor FSM



IDLE · 시작 대기

busy=0에서 start 수락
카운터와 거리 값 초기화

START · TRIG 생성

TRIG HIGH 10 µs 유지
tick=9에서 WAIT로 전이

WAIT / DISTANCE · 측정

ECHO 상승 후 HIGH 폭 계수
하강 감지 시 거리 저장

STOP · 완료와 재시작 대기

진입 시 done 1클럭(10 ns)
10 ms 후 IDLE, busy=0

timeout · 응답 제한

WAIT·DISTANCE에서 36 ms
거리 400 저장 후 STOP

Related Technical

Temperature-Humidity Sensor (DHT-11)



정면 기준 오른쪽부터 1·2·3·번 핀

핀	역할 (4핀 단품 기준)
1 VDD	전원 3~5.5 V
2 DATA	양방향 데이터 / 풀업 저항 연결
3 GND	공통 접지

측정 원리 및 출력

- 측정 대상: 주변 공기 온도(°C)·상대습도(%RH)
- 출력 방식: DATA 1선을 통한 디지털 통신

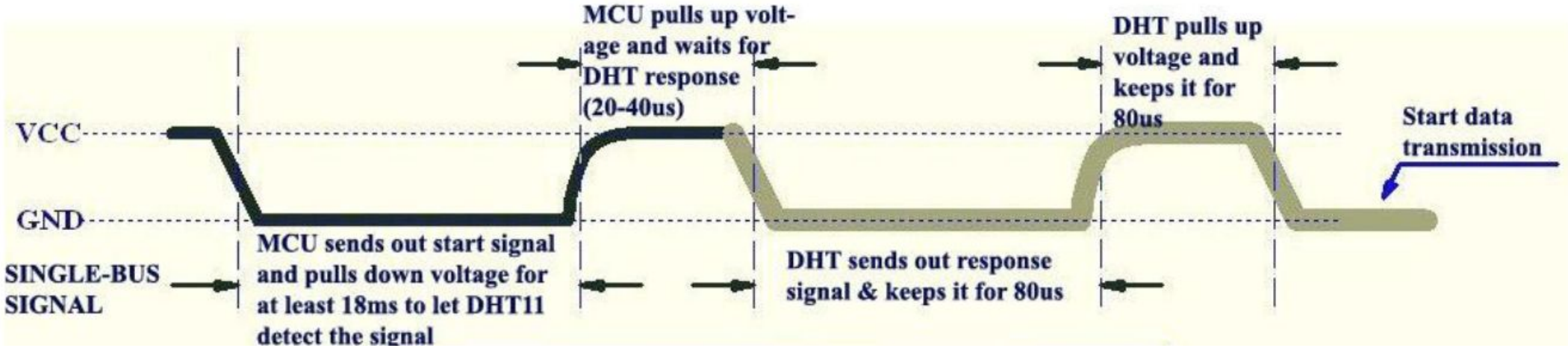
40비트 수신: 습도 16 + 온도 16 + 체크섬 8

항목	온도	상대습도
측정 범위	0~50 °C	20~90 %RH*
정확도	±2 °C	±5 %RH
분해능	1 °C	1 %RH
측정 간격	최소 1초	최소 1초

Related Technical

Temperature-Humidity Sensor (DHT-11)

한 개의 데이터 선으로 시작 신호와 센서 응답을 교환



① FPGA가 시작

LOW ≥18 ms / 구현값 19 ms

② 수신으로 전환

HIGH 20~40 µs / 구현값 30 µs

③ 센서가 응답

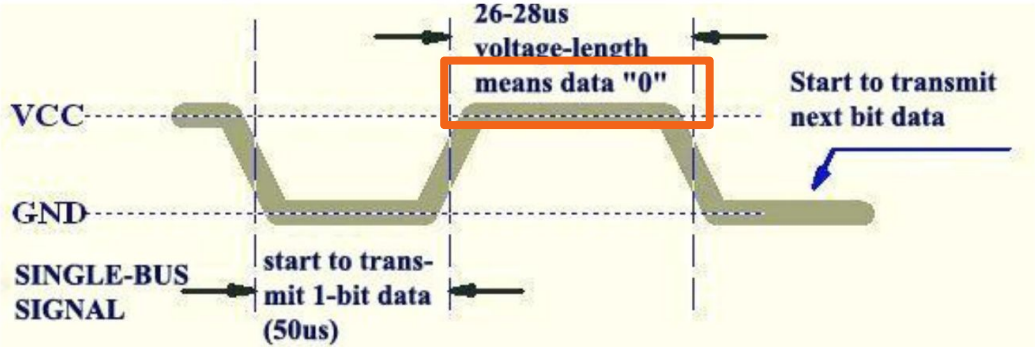
LOW 80 µs 후 HIGH 80 µs

START·WAIT에서는 FPGA가 구동, 이후 Z로 해제해 센서 응답을 수신.

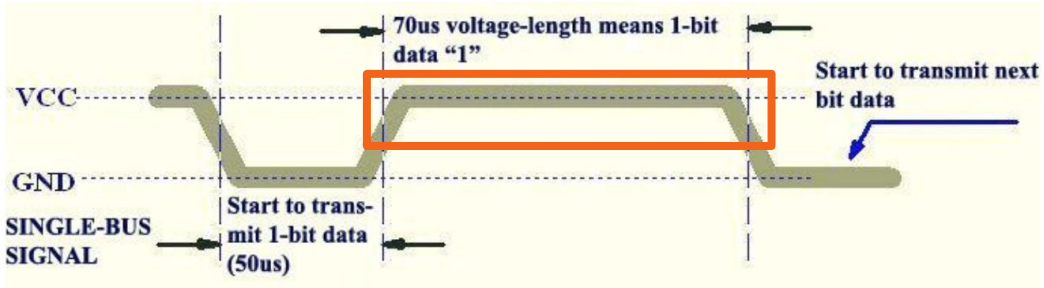
Related Technical

DHT11 데이터 형식과 판정 기준

HIGH 폭으로 0 / 1을 구분하고, 40 bit를 MSB부터 수신



0: HIGH 26~28 µs



1: HIGH 약 70 µs

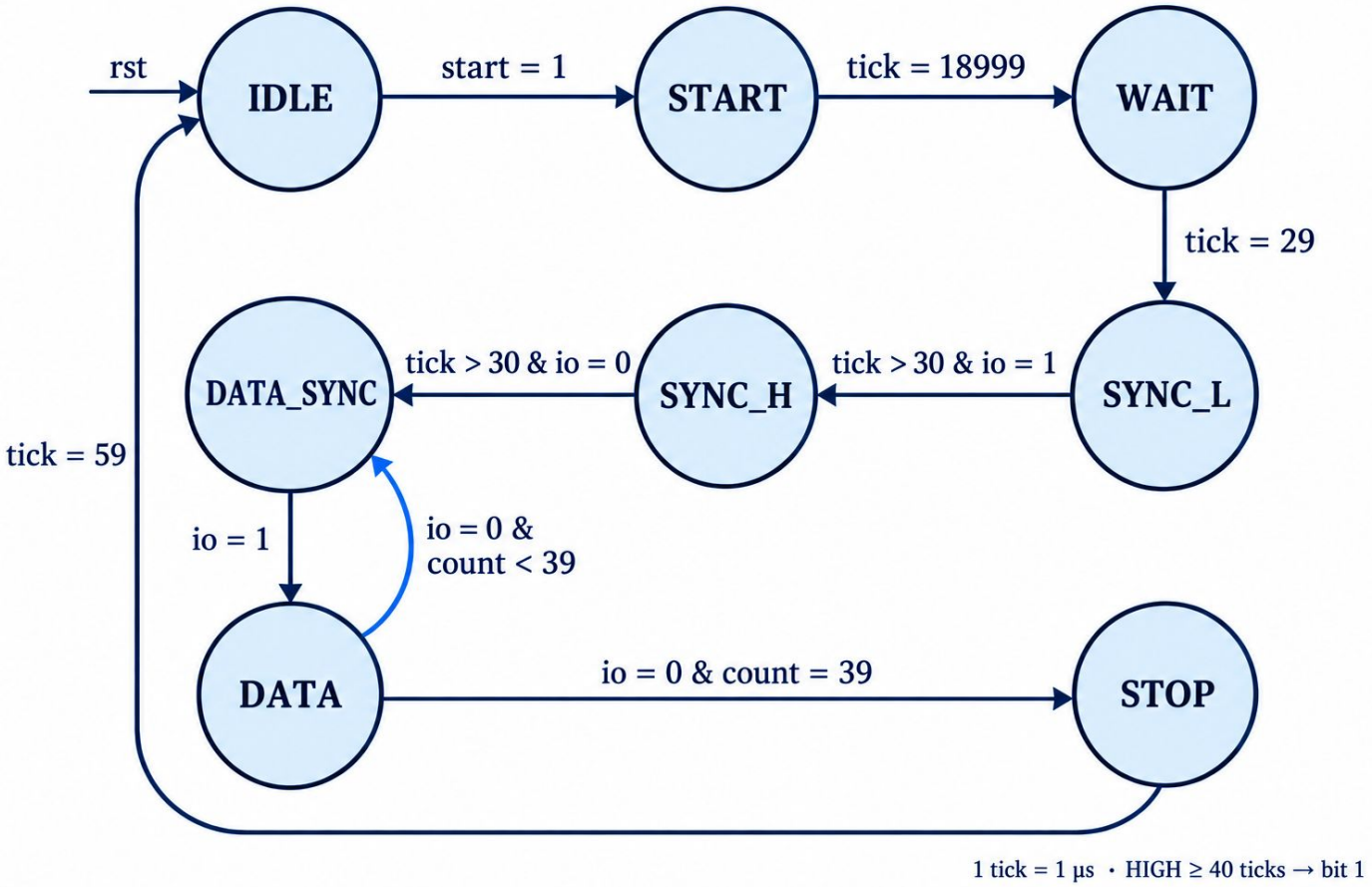
습도 정수	습도 소수	온도 정수	온도 소수	체크섬
8 bit	8 bit	8 bit	8 bit	8 bit

구현 판정: tick_cnt ≥ 40이면 1, 미만이면 0. LOW 확인 시 tick_cnt를 0으로 초기화.

체크섬: 앞 4바이트 합의 하위 8 bit와 비교. TB는 0을 30 µs, 1을 70 µs로 입력.

Related Technical

Temperature-Humidity Sensor - FSM



IDLE · 시작 대기

busy=0에서 start 수락
데이터와 카운터 초기화

START / WAIT · 시작 신호

LOW 19 ms → HIGH 30 µs
이후 출력을 Z로 해제

SYNC_L / SYNC_H · 응답

tick>30 이후 입력 레벨 확인
전이할 때 tick=0

DATA_SYNC / DATA · 수신

HIGH 폭 ≥40 µs이면 bit 1
하강 시 저장, 40 bit 반복

STOP · 수신 종료

60 µs 후 IDLE, busy=0
IDLE & valid일 때 출력 유효

시뮬레이션 시나리오

시나리오 개요

구분	목적	주요 입력	확인 항목
S1	**HC-SR04 측정 FSM 동작 검증** start · busy · 거리 계산 · done	start 펄스 (busy 중 재입력 포함) ECHO HIGH 600 µs / 1,200 µs us_tick 1 µs	TRIG 10 µs, 상태 IDLE(0) → ⋯ → STOP(4) 거리 10 cm · 20 cm 저장, done 1클럭 busy 중 재입력 무시, 10 ms 후 busy=0
S2	**DHT11 수신 FSM 동작 검증** 시작 처리 · 비트 0/1 판정	start 펄스 (busy=0 / busy=1) dht11_io 데이터 HIGH 30 µs(0) · 70 µs(1) us_tick 1 µs	IDLE→START 전이, busy 중 START 유지 DATA_SYNC→DATA, tick_cnt 1 µs 계수 tick_cnt < 40 → bit 0 저장, data_cnt 증가
S3	**ascii_sender 응답 생성 검증** 이벤트 · 모드별 ASCII 출력	fail · success · cmdNONE · status 펄스 sw1·sw0 모드별 send_data 전송 중 full = 1 (30 ns)	FAIL · DONE · NONE + 공백 24%43*C · 289cm · 3cm(0 생략) · 12:45:13:89 full 동안 전송 대기 후 이어서 출력
S4	UART status 응답 검증 top 통합 / 골든 모델 자동 비교	sw = 10, echo 123 cm 측정 후 rx “s”	rx_pop, w_status 1클럭 display “123cm “ 6바이트
S5	커맨드 → 버튼 신호 분배 검증 모드별 제어 신호 , 응답	sw = 01, rx “r” sw = 11 (busy = 0), rx “u”	stopwatch_btnR만 1 tick, DONE dht11_start 발생 안 함, NONE
S6	명령 · 버튼 동시 입력 검증 OR 병합 처리	sw = 01, rx “r” + btnR	stopwatch_btnR 1 tick RUN → STOP 1회 전환, DONE
공통 조건	clk 100 MHz (#5 토글) · 9600 bps 8N1 · 1 bit = 104,160 ns (651 × 16 tick × 10 ns)		

시뮬레이션 시나리오

S1 Ultrasonic sensor 측정 검증

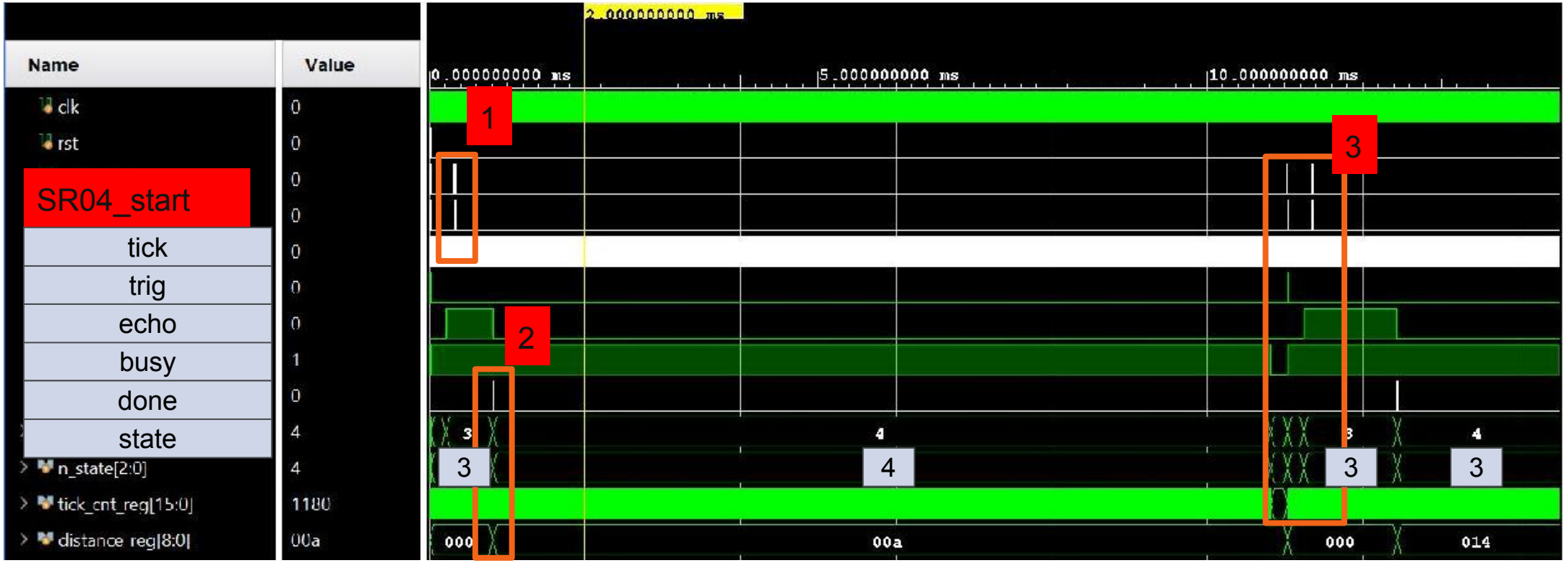
ID	검증 항목	입력 · 조건	기대 결과
1	측정 시작과 TRIG	busy=0에서 start 입력	IDLE(0) → START(1), busy=1 TRIG를 10 µs 출력한 뒤 WAIT(2) 전이
2	busy 중 재입력	START·DISTANCE에서 start 재입력	진행 중인 측정을 유지 추가 TRIG 및 측정 재시작 없음
3	ECHO 폭 계수	ECHO HIGH 600 µs 입력 w_us_tick 주기 1 µs	WAIT(2) → DISTANCE(3) tick_cnt_reg가 1 µs마다 증가
4	거리 저장과 done	ECHO 하강 후 데이터 처리 시점 확인	DISTANCE(3) → STOP(4), tick 599 → 0 distance=00A(10 cm), done=1클럭(10 ns)
5	완료 후 busy 해제	STOP 진입 후 10 ms 대기	done 종료 후에도 busy=1 유지 10 ms 경과 시 IDLE(0), busy=0
6	다음 측정과 재입력	busy=0에서 새 start ECHO 1,200 µs 중 재입력	새 측정에서 거리 초기화, busy 재입력 무시 다음 결과 014(20 cm), 총 TRIG·done 각 2회

시뮬레이션 결과

S1 · start 재입력과 연속 측정

결과 요약 busy=1 재입력은 무시 · 거리 저장·done 발생 · busy=0 복귀 후 새 start 수락

① 측정 중 재입력 ② 거리 저장·done ③ busy 해제 → 새 start → busy 중 재입력



00A=10 cm → 새 start에서 000 → 다음 결과 014=20 cm. STOP의 10 ms 대기까지 같은 시간축에 표시.

시뮬레이션 결과

S1 · 데이터 저장 시 tick·state 변화

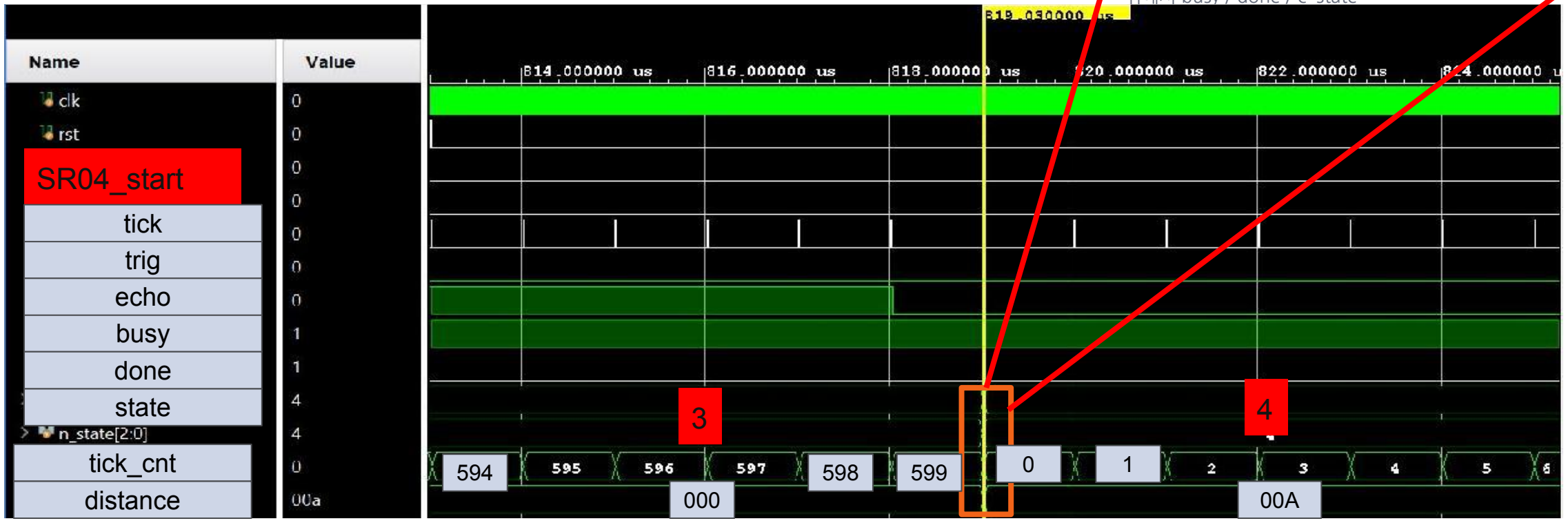
결과 요약

ECHO 하강 확인 → DISTANCE(3) → STOP(4)
tick 599→0 · distance 000→00A · busy=1 유지

done 확대 · 819.025~819.035 μs (10 ns)



위에서 busy / done / c_state



시뮬레이션 시나리오

S2 Temperature-Humidity Sensor 측정 검증

ID	검증 항목	입력 · 조건	기대 결과
1	대기 중 시작 수락	busy=0, IDLE(0)에서 start 입력	IDLE(0) → START(1), busy=1 시작 신호 출력 및 측정 진행
2	busy 중 시작 무시	busy=1, START(1)에서 start 재입력	START(1) 유지 진행 중인 시작 신호와 tick 계수 유지
3	데이터 HIGH 진입	DATA_SYNC(5)에서 dht11_io 상승	DATA_SYNC(5) → DATA(6) tick_cnt 초기화 후 HIGH 폭 계수 시작
4	HIGH 시간 계수	DATA(6), dht11_io=1 us_tick 주기 1 μs	DATA(6)를 유지하며 tick_cnt 증가 입력 HIGH 시간을 1 μs 단위로 측정
5	비트 0 저장	tick_cnt < 40에서 dht11_io 하강	비트 0 저장, data_cnt 32 → 33 DATA(6) → DATA_SYNC(5), tick_cnt=0

시뮬레이션 결과

S2

결과 요약 **busy = 0에서 start 입력 시 IDLE(0) → START(1) 전이, busy = 1에서 start 재입력 시 START(1) 유지 확인**

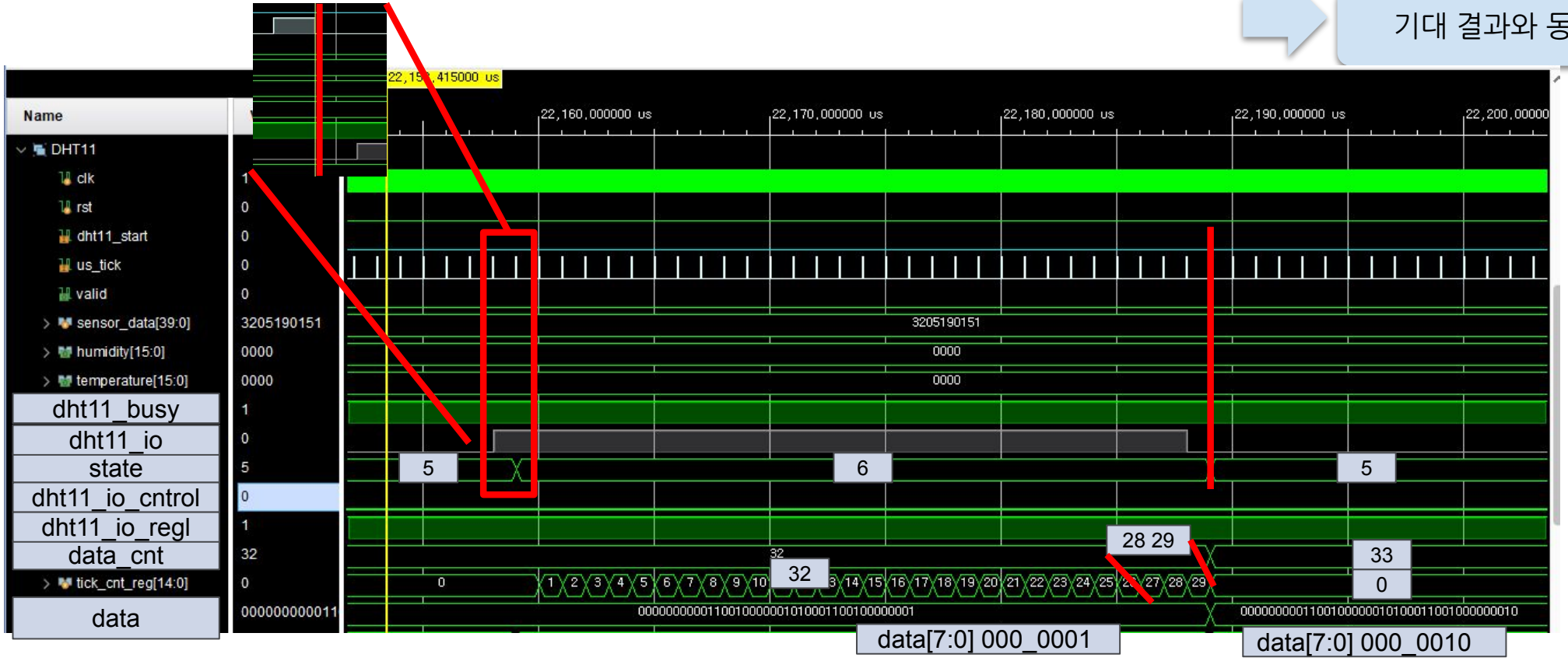
기대 결과와 동일 -> 정상 작동



시뮬레이션 결과

S2

결과 요약 **dht11_io 상승 시 DATA_SYNC(5) → DATA(6) 전이, 하강 감지 시 tick_cnt < 40 조건으로 비트 0 저장 및 data_cnt 32 → 33 증가 확인**



시뮬레이션 시나리오

S3 Control_Unit 의 ascci 출력 데이터 검증

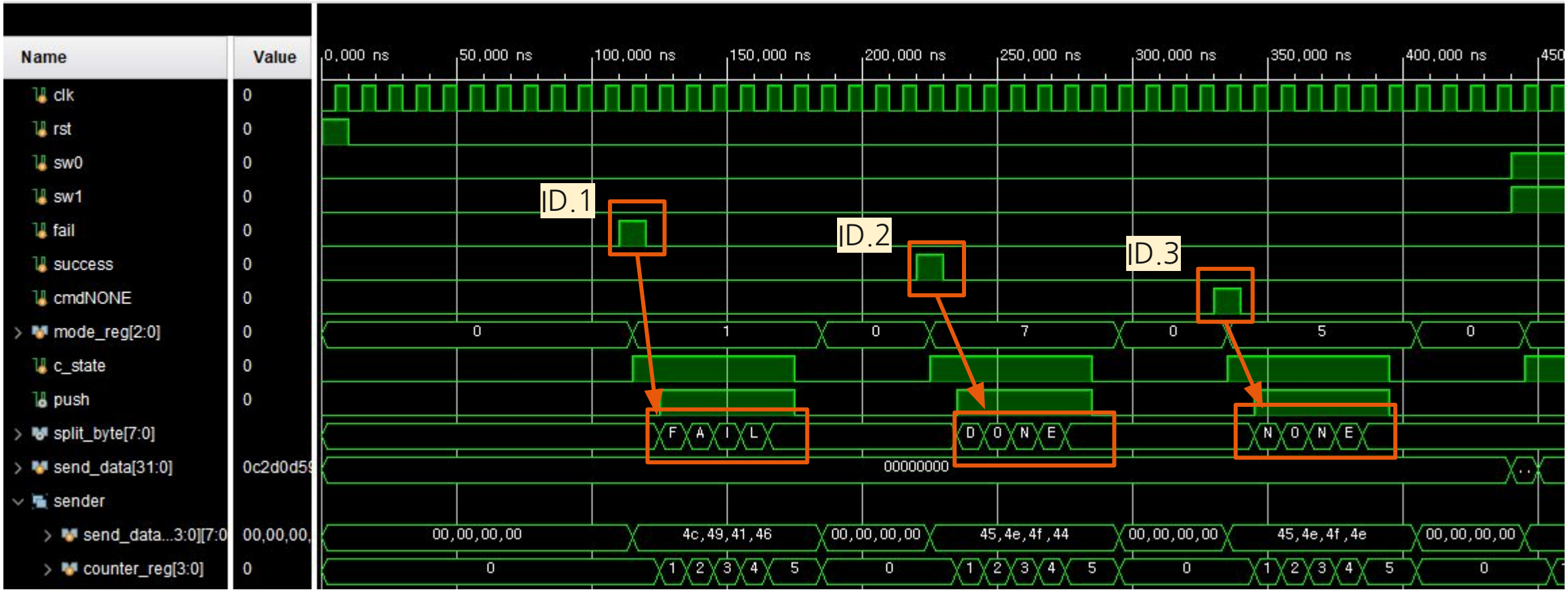
ID	검증 항목	입력 · 조건	기대 결과
	리셋 후 대기 상태	rst = 1 → 0, 이벤트 입력 없음	IDLE(c_state = 0), push = 0, sender_busy = 0
1	실패 응답	fail 1클럭 펄스	FAIL + 공백 5바이트 순서대로 push 후 IDLE 복귀
2	성공 응답	success 1클럭 펄스	DONE + 공백 5바이트 push
3	미지원 명령 응답	cmdNONE 1클럭 펄스	NONE + 공백 5바이트 push
4	DHT11 값 전송	sw1·sw0 = 11, status, {24, 45, 43, 89}	24%43*C + 공백 8바이트 (습도 24 %, 온도 43 °C)
5	SR04 거리 전송	sw1·sw0 = 10, status, {0, 0, 2, 89}	289cm + 공백 6바이트
6	SR04 앞자리 0 생략	sw1·sw0 = 10, status, {0, 0, 0, 3}	3cm + 공백 4바이트 (백·십의 자리 0은 전송 안 함)
7	스톱워치 값 전송	sw1·sw0 = 01, status, {12, 45, 13, 89}	12:45:13:89 + 공백 12바이트
8	시계 값 전송	sw1·sw0 = 00, status, {12, 45, 13, 89}	12:45:13:89 + 공백 12바이트
9	TX FIFO full 대기	시계 전송 중 full = 1을 30 ns 유지	full 동안 다음 바이트 대기, 해제 후 이어서 전송 · 누락·중복 없음

시뮬레이션 결과

S3 ascii decoer 출력 검증

결과 요약 입력에 따라 fail,done,none 출력

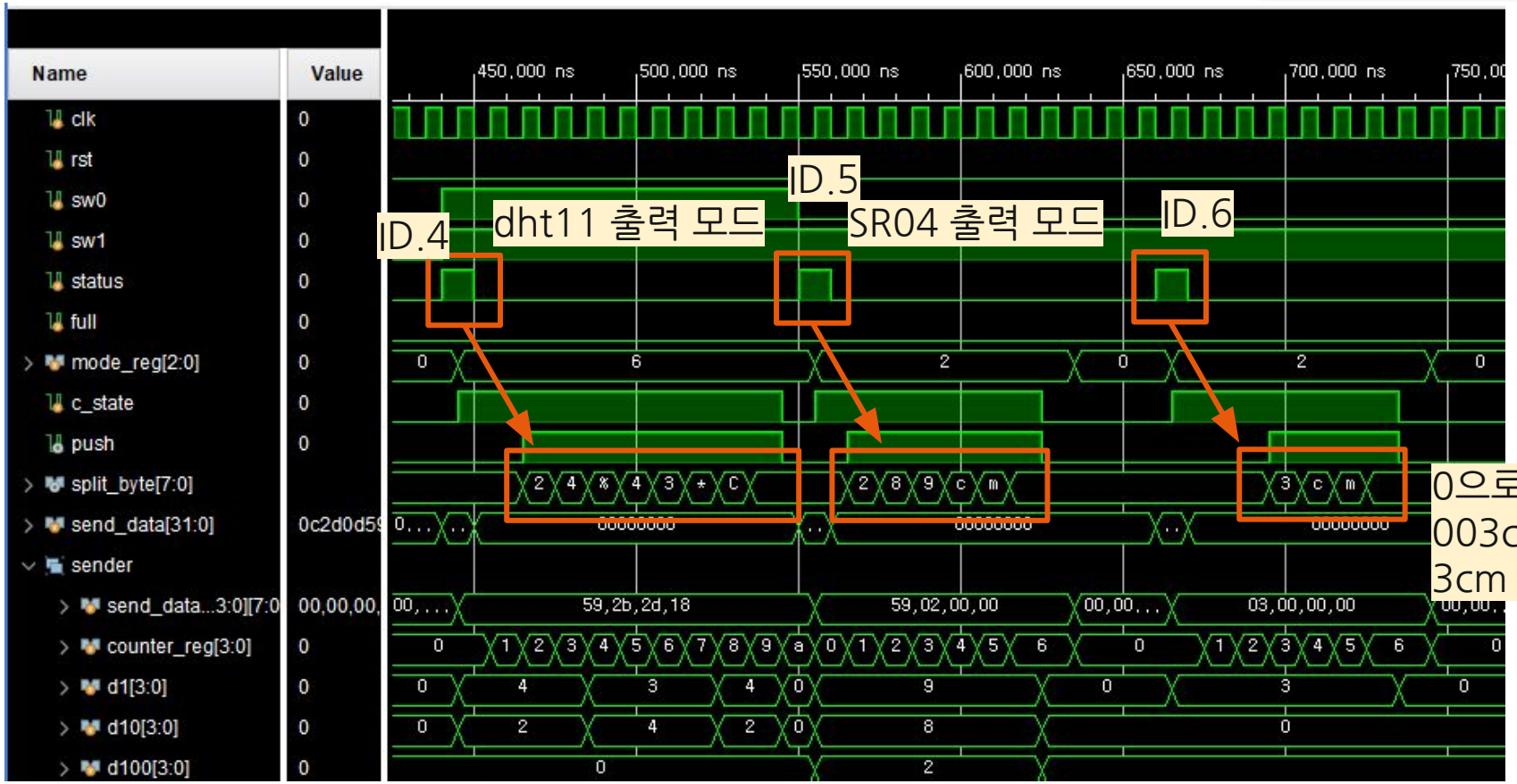
기대결과와 동일 -> 정상 작동



시뮬레이션 결과

S3 ascii decoer 출력 검증

결과 요약 status 입력시 모드에 따른 값 출력 → 기대 결과와 동일 -> 정상 작동

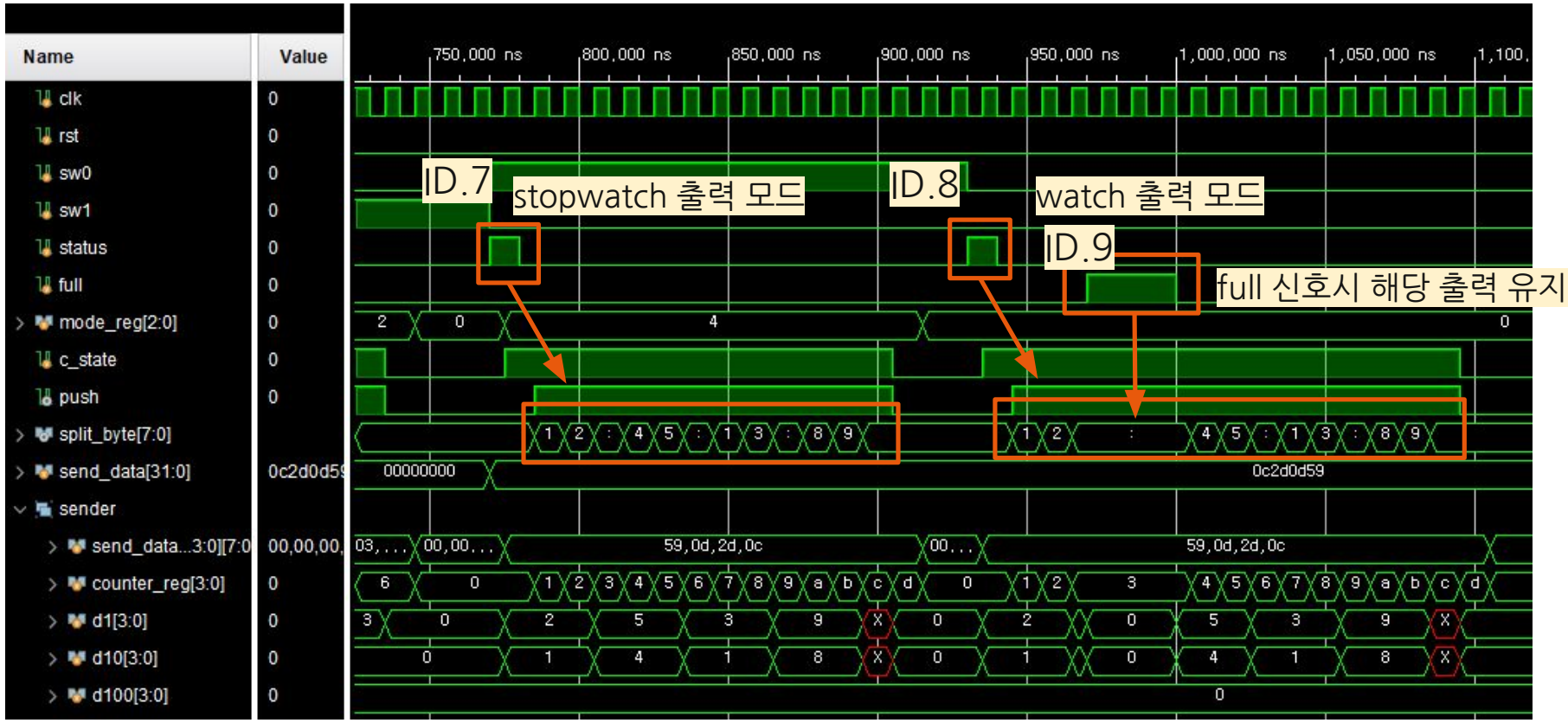


0으로 시작시 생략
003cm -> X
3cm -> O

시뮬레이션 결과

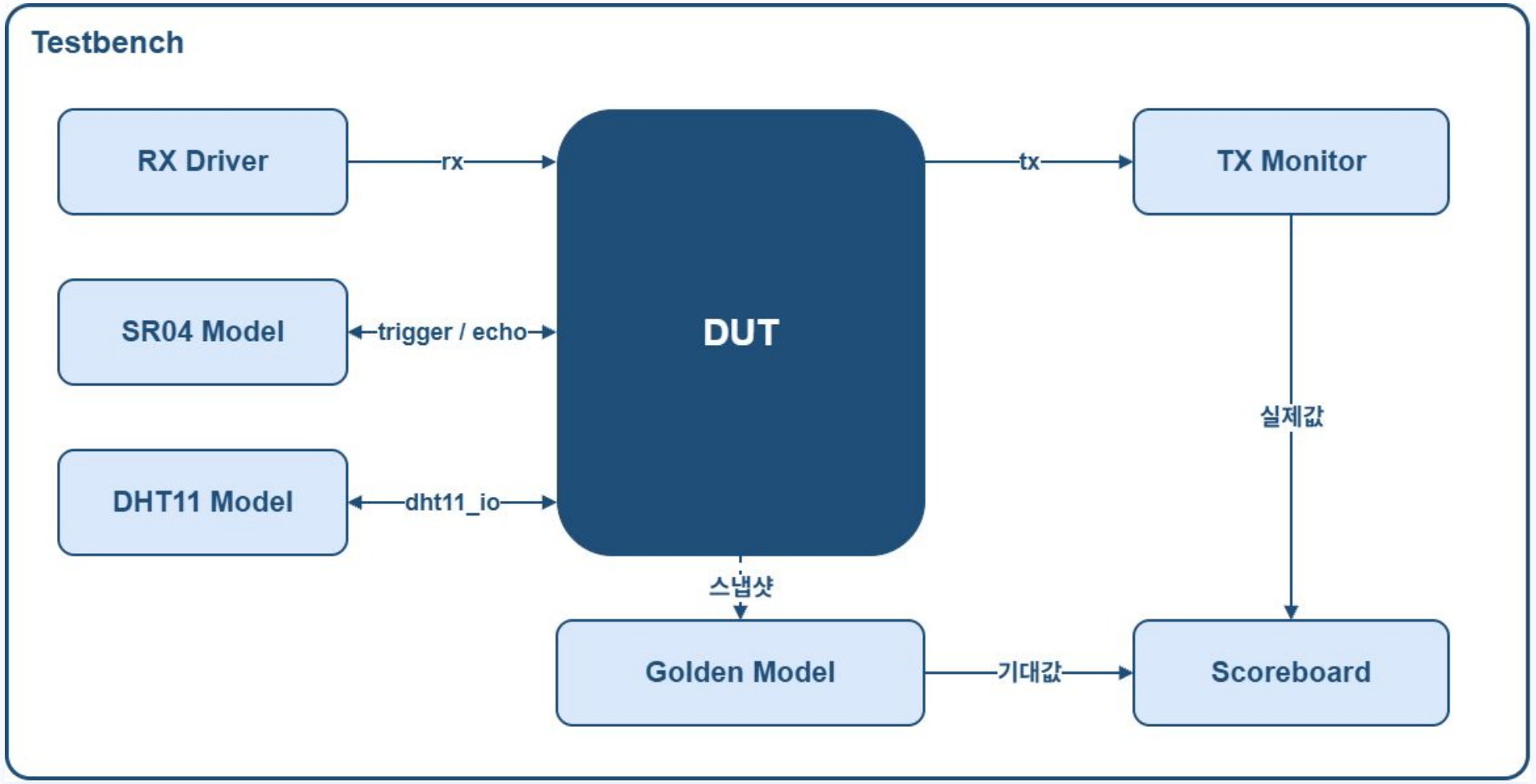
S3 ascii decoer 출력 검증

결과 요약 status 입력시 stopwatch,watch 출력 & full 신호시 해당 출력 유지 → 기대 결과와 동일 ->정상 작동



시뮬레이션 시나리오

Top Module Testbench 구성



시뮬레이션 시나리오

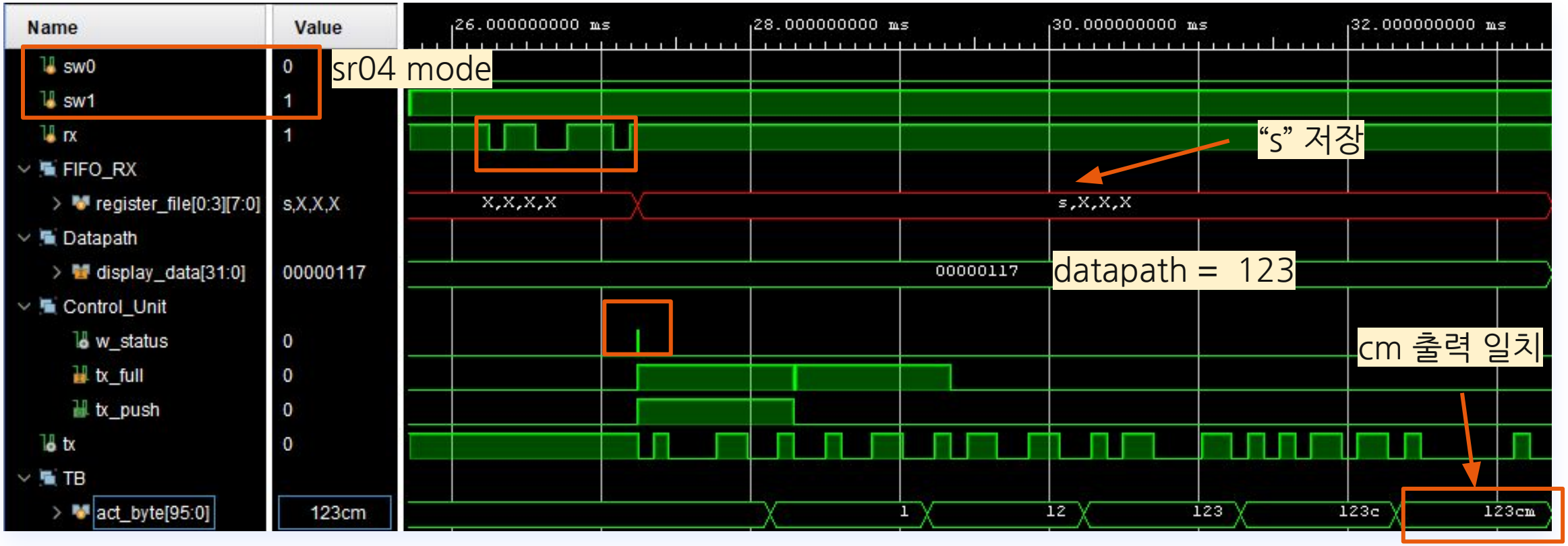
S4 “s” 커맨드 응답 검증

ID	검증 항목	입력 · 조건	기대 결과
1	명령 수신	rx로 “s” (0x73) 수신	rx_pop 1클럭, w_status = 1 (다른 명령 신호는 0)
2	응답 문자열 생성	w_status = 1	display_data 캡쳐 후 “123cm “ 6바이트 TX에 push
3	TX FIFO full 처리	6byte > depth 4	tx_full 동안 마지막 바이트 대기(tx_push 유지)
4	비교 판정	골든 모델 vs TX 수신	act_byte = “123cm “, Tcl Console PASS 문구

시뮬레이션 결과

S4 “s” 커맨드 응답 검증 - overall

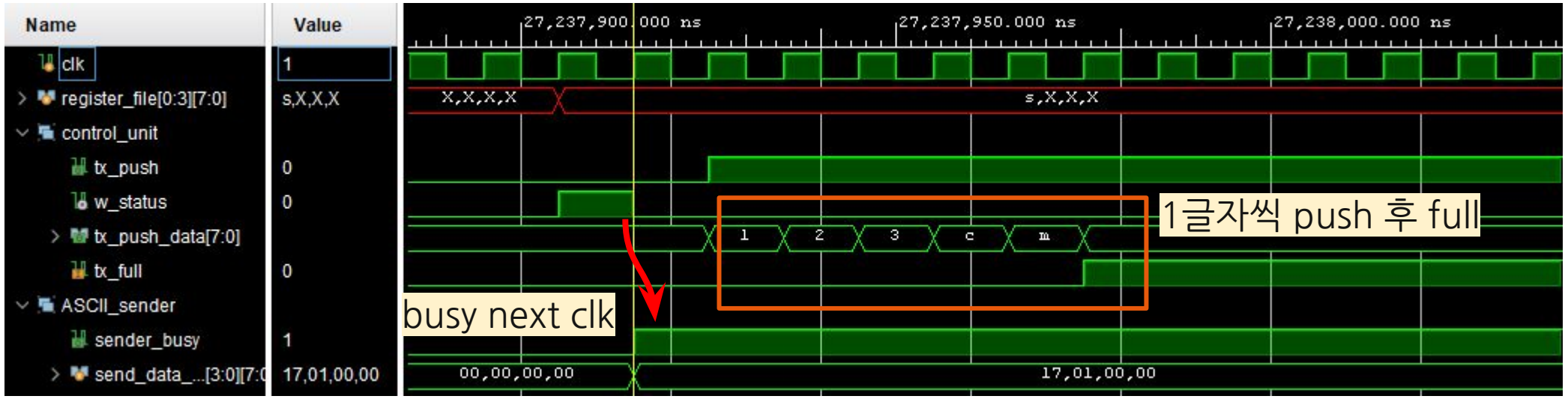
결과 요약 “s” → “123cm” 출력 → 전체 동작 결과 정상



시뮬레이션 결과

S4 “s” 커맨드 응답 검증 - zoom

결과 요약 busy, full 타이밍 맞음 → **동작 결과 정상**




```
[RX DRIVER ] send 's'
[GOLDEN    ] cmd 's', snapshot 0x00000117
[SCOREBOARD] PASS #0 exp '1' got '1'
[SCOREBOARD] PASS #1 exp '2' got '2'
[SCOREBOARD] PASS #2 exp '3' got '3'
[SCOREBOARD] PASS #3 exp 'c' got 'c'
[SCOREBOARD] PASS #4 exp 'm' got 'm'
[SCOREBOARD] PASS #5 exp ' ' got ' '
[RESULT    ] PASS : 6 bytes checked, 0 error
```

시뮬레이션 시나리오

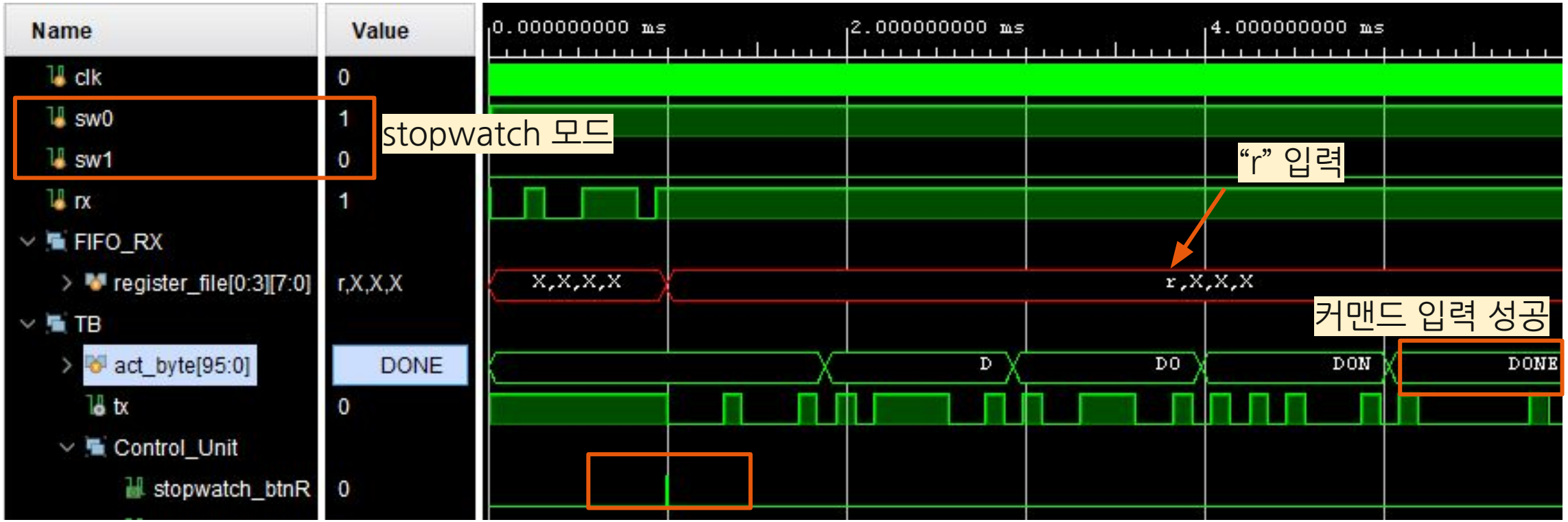
S5 UART 커맨드 분배

ID	검증 항목	입력 · 조건	기대 결과
1	해당 모드로만 분배	sw = 01, “r” 수신	stopwatch_btnR = 1 watch_btn , sr04_start , dht11_start = 0
2	1 tick 펄스로 상태 전환	stopwatch_btnR 1클럭	STOP → RUN 1회 전환, 카운트 시작
3	성공 응답	w_cmd_success = 1	“DONE “ 송신, err_cnt = 0

시뮬레이션 결과

S5 UART 커맨드 분배

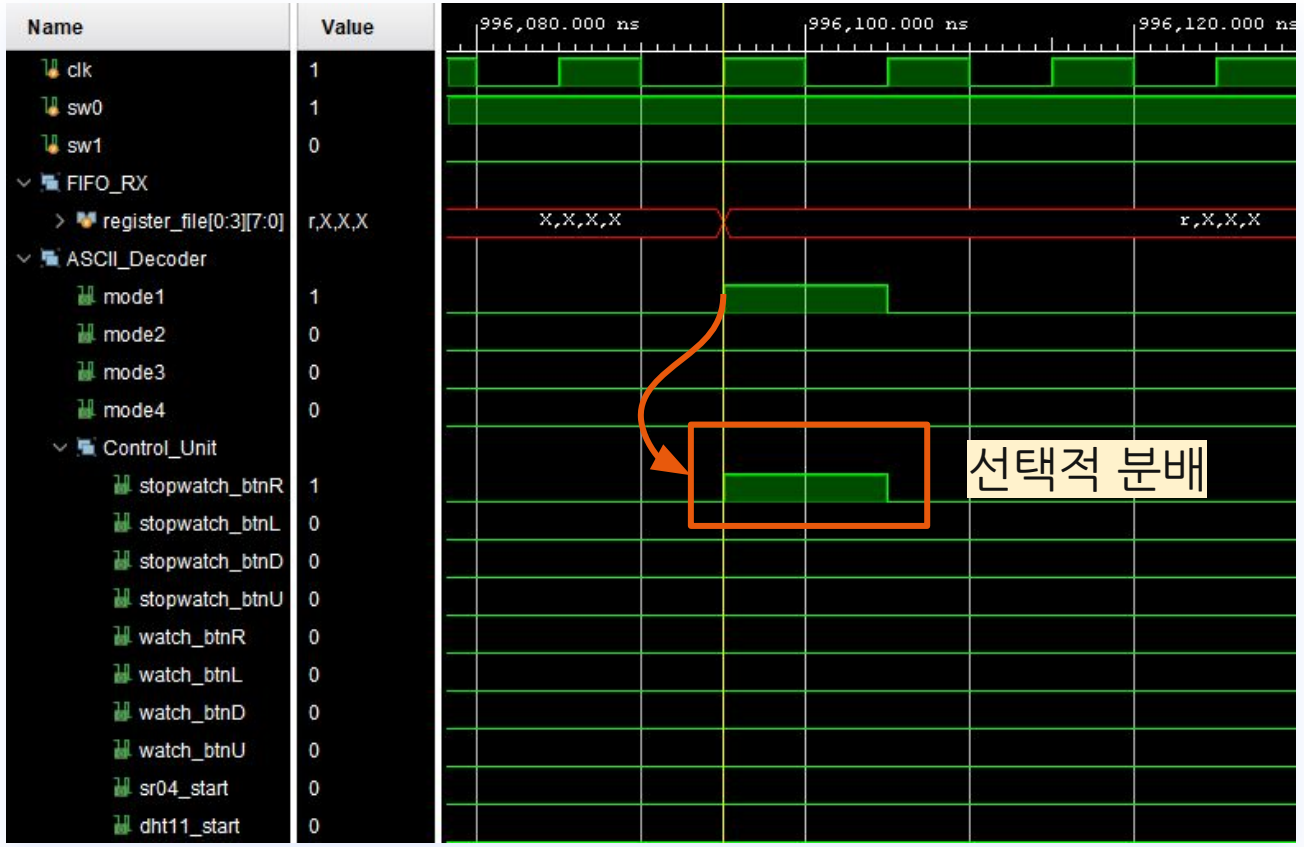
결과 요약 “r” → stopwatch_btnR 1 tick, “DONE” 응답 ⇒ **전체 동작 결과 정상**



시뮬레이션 결과

S5 UART 커맨드 분배 - stopwatch

결과 요약 나머지 컨트롤 출력 0 → 동작 결과 정상



시뮬레이션 시나리오

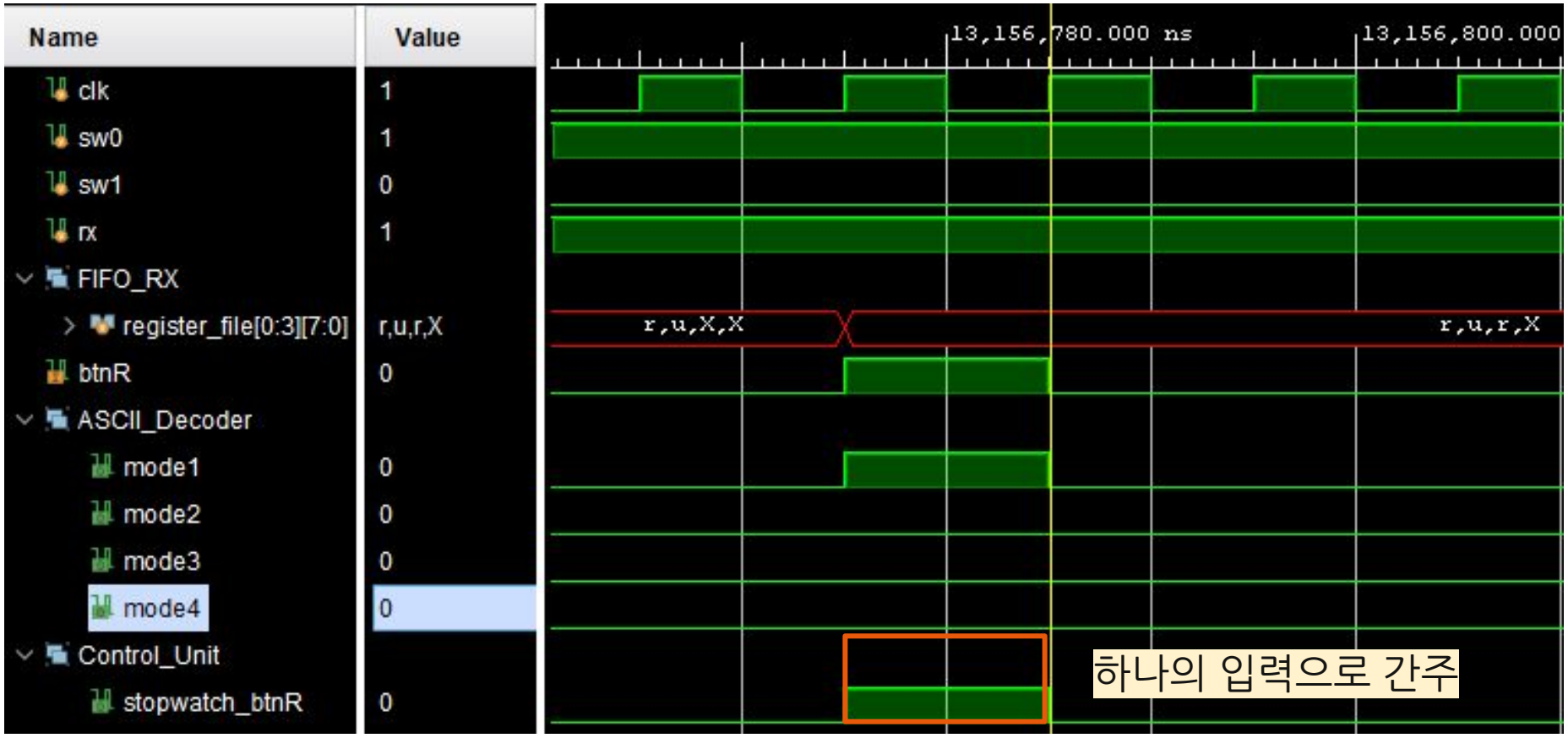
S6 UART 커맨드 & 버튼 동시 입력

ID	검증 항목	입력 · 조건	기대 결과
1	두 입력 OR 병합	sw = 01, “r” 수신 + btnR 같은 클럭	w_cmdR = 1, btnR = 1 stopwatch_btnR 1 tick
2	중복 동작 없음	stopwatch_btnR 1클럭	RUN → STOP 1회만 전환
3	성공 응답	w_cmd_success = 1	“DONE “ 송신, err_cnt = 0

시뮬레이션 결과

S6 커맨드 & 버튼 동시입력

결과 요약 stopwatch_btnR 단일 동작 → 동작 결과 정상



시뮬레이션 시나리오

기능 검증 Checklist

ID	검증 항목	입력 · 조건	기대 결과
1	시계 수정 r / l / u / d	sw = 00, sw3 = 1	r · l: 수정 자리 이동 u · d: 선택 자리 +1 / −
2	스톱워치 r / l / u / d	sw = 01	r: RUN ↔ STOP, l: 초기화 u: up / down 전환, d: 랩타임
3	표시 모드 전환	{sw1, sw0} = 00 ~ 11	시계 / 스톱워치 / SR04 / DHT11 순서로 FND 표시
4	표시 자리 전환	sw2 = 0 / 1 (시계 , 스톱워치)	0: 초 : 1/100초 1: 시 : 분
5	busy 중 버튼 무시	sw = 10 / 11, busy = 1, btnR	start 발생 안 함, 측정 유지

Implementation

Timing Closure

결과 요약 **WNS 및 WHS 모두 양수 달성, Timing Violation 없음**

<WNS (Worst Negative Slack)>

2.157 ns

Required time – Arrival time

15.216 ns – 13.058 ns

BUFGCTRL_X0Y0	BUFG (Prop_bufg_I_O)	0.091	13.341 r	clk_IBUF_BUFG_inst/O
	net (fo=576, routed)	1.600	14.941	u_DATAPATH/u_DHT11_TOP/u_DHT11_CONTROL/clk_IBUF_BUFG
SLICE_X17Y122	FDCE		r	u_DATAPATH/u_DHT11_TOP/u_DHT11_CONTROL/FSM_sequential_c_state_reg[2]/C
	clock pessimism	0.281	15.222	
	clock uncertainty	-0.035	15.187	
SLICE_X17Y122	FDCE (Setup_fdce_C_D)	0.029	15.216	u_DATAPATH/u_DHT11_TOP/u_DHT11_CONTROL/FSM_sequential_c_state_reg[2]

required time	15.216
arrival time	-13.058
slack	2.157

<WHS (Worst Hold Slack)>

0.037 ns

Arrival time – Required time

1.721 ns – 1.684 ns

BUFGCTRL_X0Y0	BUFG (Prop_bufg_I_O)	0.029	1.128 r	clk_IBUF_BUFG_inst/O
	net (fo=576, routed)	0.912	2.040	u_UART/U_FIFO_RX/U_REGISTER_FILE/register_file_reg_0_3_0_5/WCLK
SLICE_X14Y117	RAMD32		r	u_UART/U_FIFO_RX/U_REGISTER_FILE/register_file_reg_0_3_0_5/RAMA/CLK
	clock pessimism	-0.503	1.537	
SLICE_X14Y117	RAMD32 (Hold_ramd32_CLK_I)			
		0.147	1.684	u_UART/U_FIFO_RX/U_REGISTER_FILE/register_file_reg_0_3_0_5/RAMA

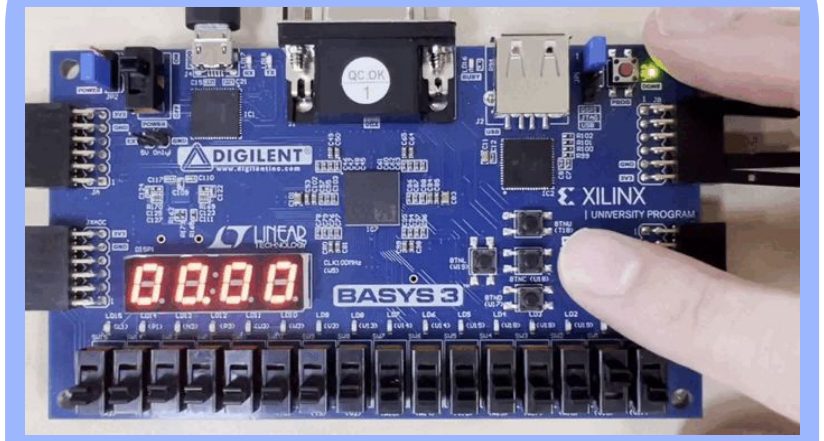
required time	-1.684
arrival time	1.721
slack	0.037

동작 영상

Button Switch Control – SR04 / DHT11

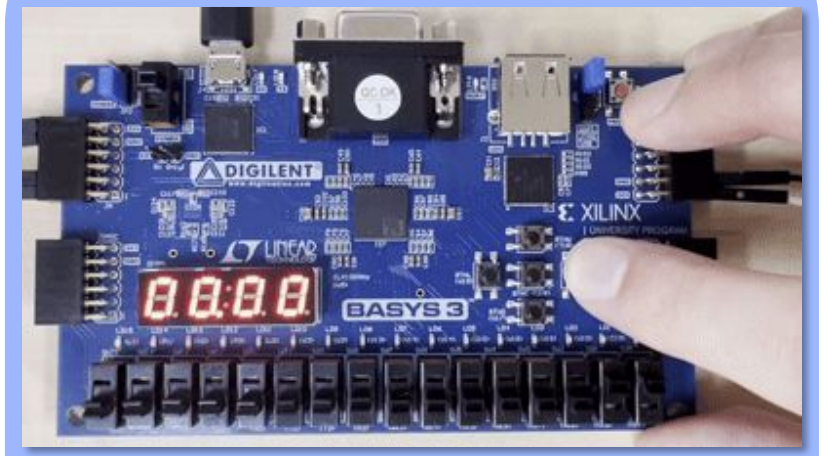
결과 요약 기존의 **Blackbox** 형태와 같이, 버튼 스위치 입력을 통한 **SR04/DHT11** 거리 측정 및 습도/온도 측정 정상 동작 검증 완료

<SR04>



- SR04 Mode (SW = 10)에서 R 버튼 클릭 시 거리 측정 정상 동작 확인

<DHT11>



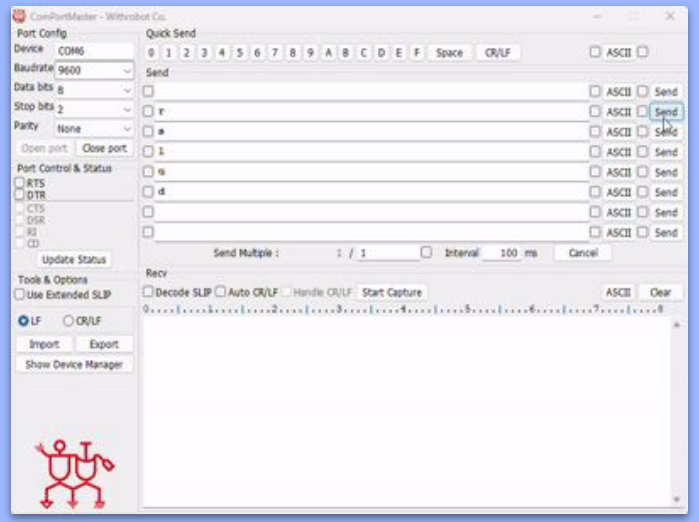
- DHT11 Mode (SW = 11)에서 R 버튼 클릭 시 습도 / 온도 측정 정상 동작 확인

동작 영상

UART ASCII Control – SR04 / DHT11

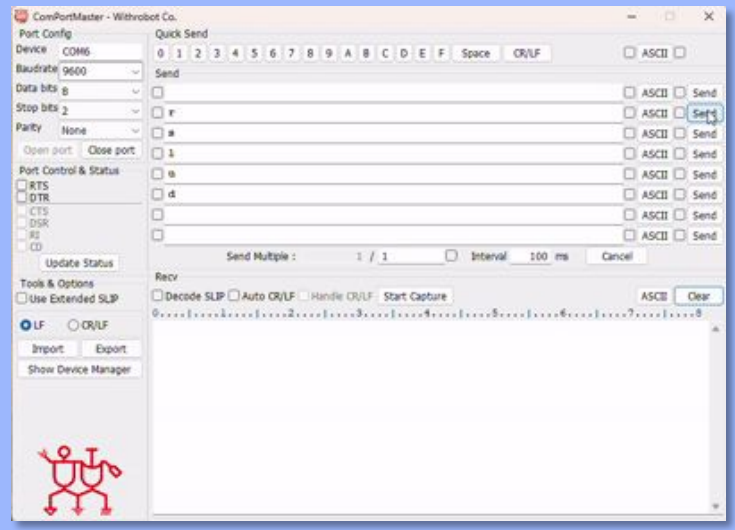
결과 요약 **UART ASCII Interface를 통한 SR04 / DHT11 거리 측정 및 습도 / 온도 측정 정상 동작 검증 완료**

<SR04>



- SR04 Mode (SW = 10)에서 ‘r’ 전송 시 거리 측정, ‘s’ 전송 시 거리 값 출력 정상 동작 확인

<DHT11>



- DHT11 Mode (SW = 11)에서 ‘r’ 전송 시 습도 / 온도 측정, ‘s’ 전송 시 습도 / 온도 값 출력 정상 동작 확인

결론

결론 구현 성과

01 Multi-System Integration 수행 완료

- FIFO-UART 인터페이스 기반 ASCII 인코딩/디코딩 방식을 적용하여 Watch, Stopwatch, SR04/DHT11 센서 프로세싱 제어 통합
- 멀티 모드 인터페이스 간 유기적 연동 및 시스템 통합 완료

02 Sensor & ASCII Decoder / Sender FSM 설계 완수

- FIFO-UART 수신 ASCII 데이터의 복호화(Decoding) 및 데이터패스 제어를 통한 안정적인 PC 연동 전송 구현
- 인코딩/디코딩 통신 프로토콜을 구축하여 기초적인 임베디드 컴퓨팅 구조 완성

03 시뮬레이션 & FPGA Integration을 통한 성공적 검증

- 6가지 시뮬레이션 시나리오를 활용한 엣지 케이스(Edge Case) 및 타이밍 무결성 검증 완수
- 실제 FPGA 보드 온보딩(Integration) 테스트를 통해 잠재적 결함 제거 및 최종 하드웨어 동작 검증 완료

고찰

고찰 개선 사항

01 실시간 처리 시스템으로 개선

- 현재 SR04 및 DHT11 FSM은 수동 버튼 입력(Start 신호) 기반으로 동작하는 제약 존재
- 타이머 기반 자동 펄스 생성 및 실시간 연속 데이터 스트리밍 처리 구조로 전환 예정

02 Pipelining으로 타이밍 마진 개선

- 현재 SR04 FSM 거리 계산 로직: 1μs Tick Counter를 분주, 100cm / 1cm 단위의 단계별 연산 적용
- DISTANCE와 STOP 상태 사이에 CALC Stage 및 파이프라인 레지스터(Pipeline Register)를 삽입하여 크리티컬 패스(Critical Path) 단축 및 타이밍 마진 확보

03 복잡한 연산을 수행하는 고차원 컴퓨팅으로 업그레이드

- 현재 FIFO-UART 인터페이스 기반의 단순 ASCII Decoder / Sender 입출력 제어 수준
- ALU(산술논리연산장치) 및 명령어 Decoder를 통합하여 복잡한 사칙연산 및 다중 명령어를 처리하는 프로세서 아키텍처로 확장

THANK YOU

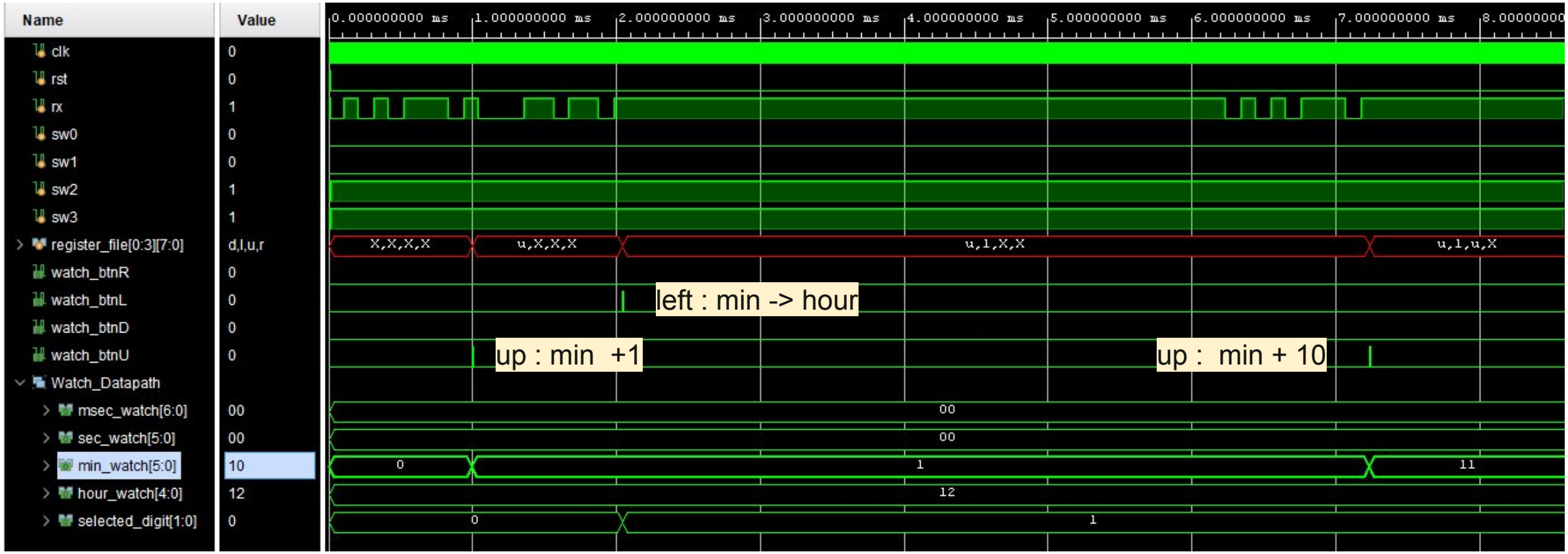
감사합니다

질의응답 · Q&A



참고 자료 - check list 시뮬레이션

WATCH 수정모드 CMD - [R , L ,U , D]



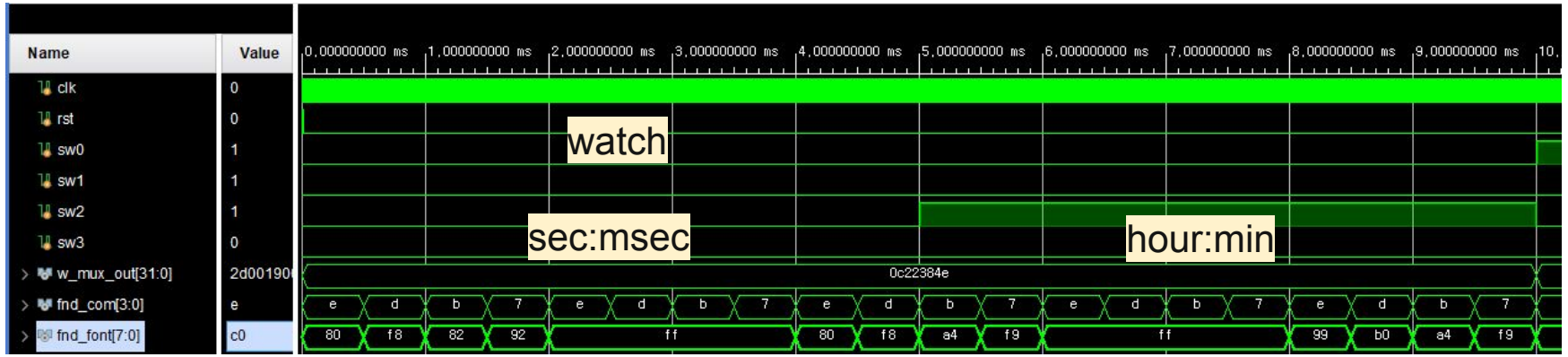
참고 자료 - check list 시뮬레이션

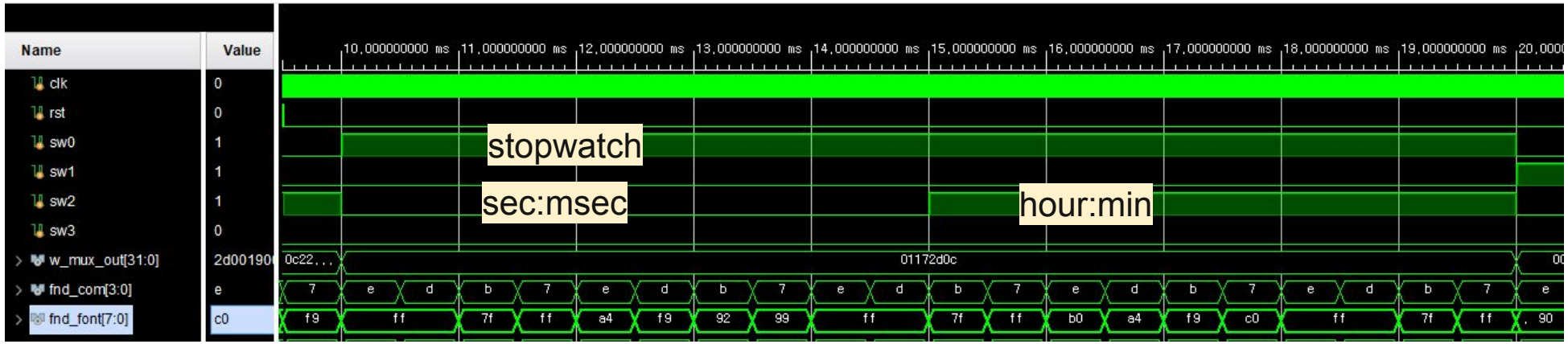
STOPWATCH CMD - [R , L ,U , D]



참고 자료 - check list 시뮬레이션

7SEGMENTS 디스플레이 모드 전환





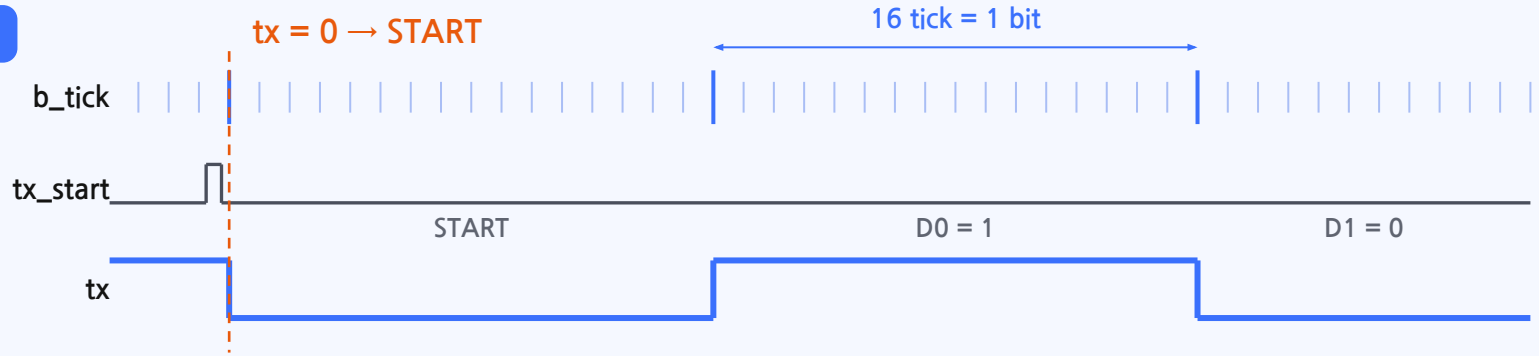
참고 자료

UART Timing — TX / RX

RX는 Baud Period 중앙에서 16 tick마다 샘플링

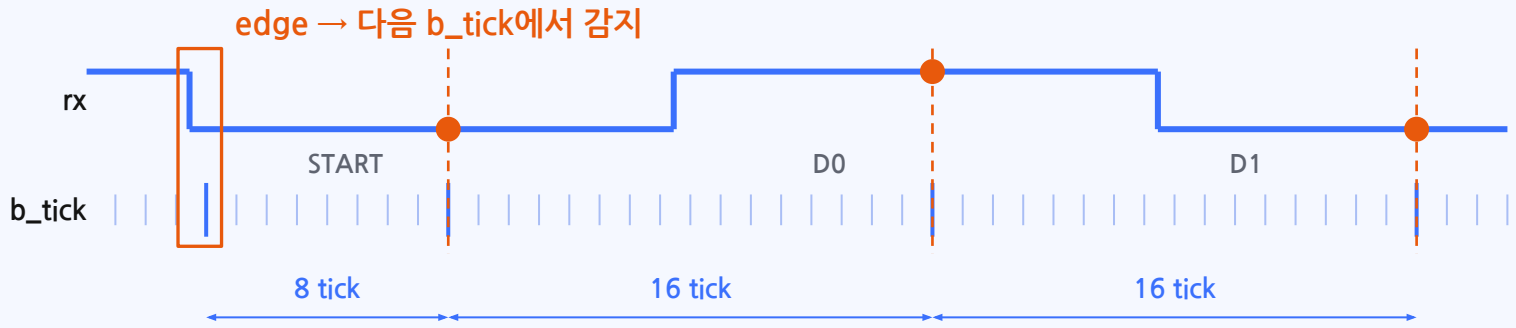
9600 bps · 16배 오버샘플링 · data = 0x41 ('A')

TX



uart_tx : tx_start 후 첫 b_tick에 tx = 0, 이후 16 tick마다 다음 비트(LSB부터) 출력

RX



uart_rx : START에서 8 tick(cnt = 7) 후 비트 중앙 → DATA에서 16 tick(cnt = 15)마다 샘플

왜 16배 오버샘플링?

1 클럭 공유가 없음

수신 측은 비트 경계를 모름 → 1 bit를 16칸으로 쪼개 START 엣지를 1/16 bit 정밀도로 찾음

2 비트 중앙에서 읽기

엣지 후 8 tick = 비트 중앙. 앞뒤로 8 tick씩 여유가 생겨 보레이트 오차 · 노이즈에 강함

3 8배보다 넉넉한 여유

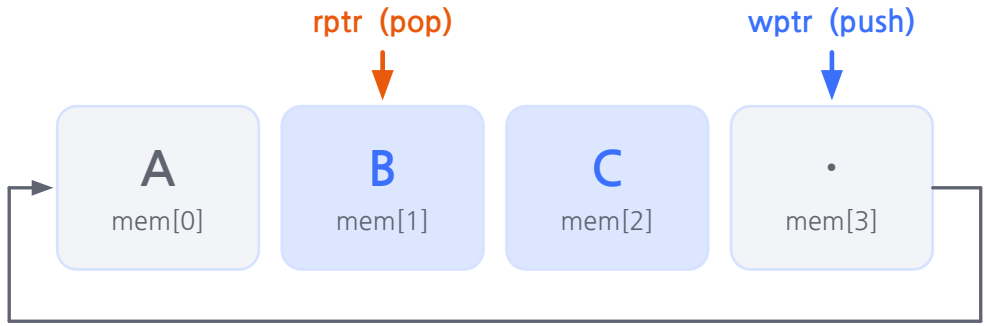
8배는 중앙까지 4 tick뿐. 16배는 오차 허용폭이 커서 가장 널리 쓰임

100 MHz ÷ (9600 × 16) ≈ 651

참고자료

FIFO (First In, First Out)

먼저 들어온 데이터가 먼저 나가는 버퍼 — 쓰는 쪽과 읽는 쪽의 속도/타이밍이 다를때 데이터를 보관



마지막 칸 다음은 0번 칸 (원형 버퍼) · 데이터 대신 포인터만 이동

이 설계의 FIFO

- register_file : 깊이 4 × 8bit 메모리
- control_unit : 2bit 포인터 wptr · rptr, full · empty 플래그
- 쓰기는 클럭 동기 (we = ~full & push)
- 읽기는 조합 출력 (pop_data = mem[rptr])

1 push (쓰기)

- wptr 칸에 push_data 저장 후 wptr + 1
- full이면 쓰기를 막아 덮어쓰기 없음

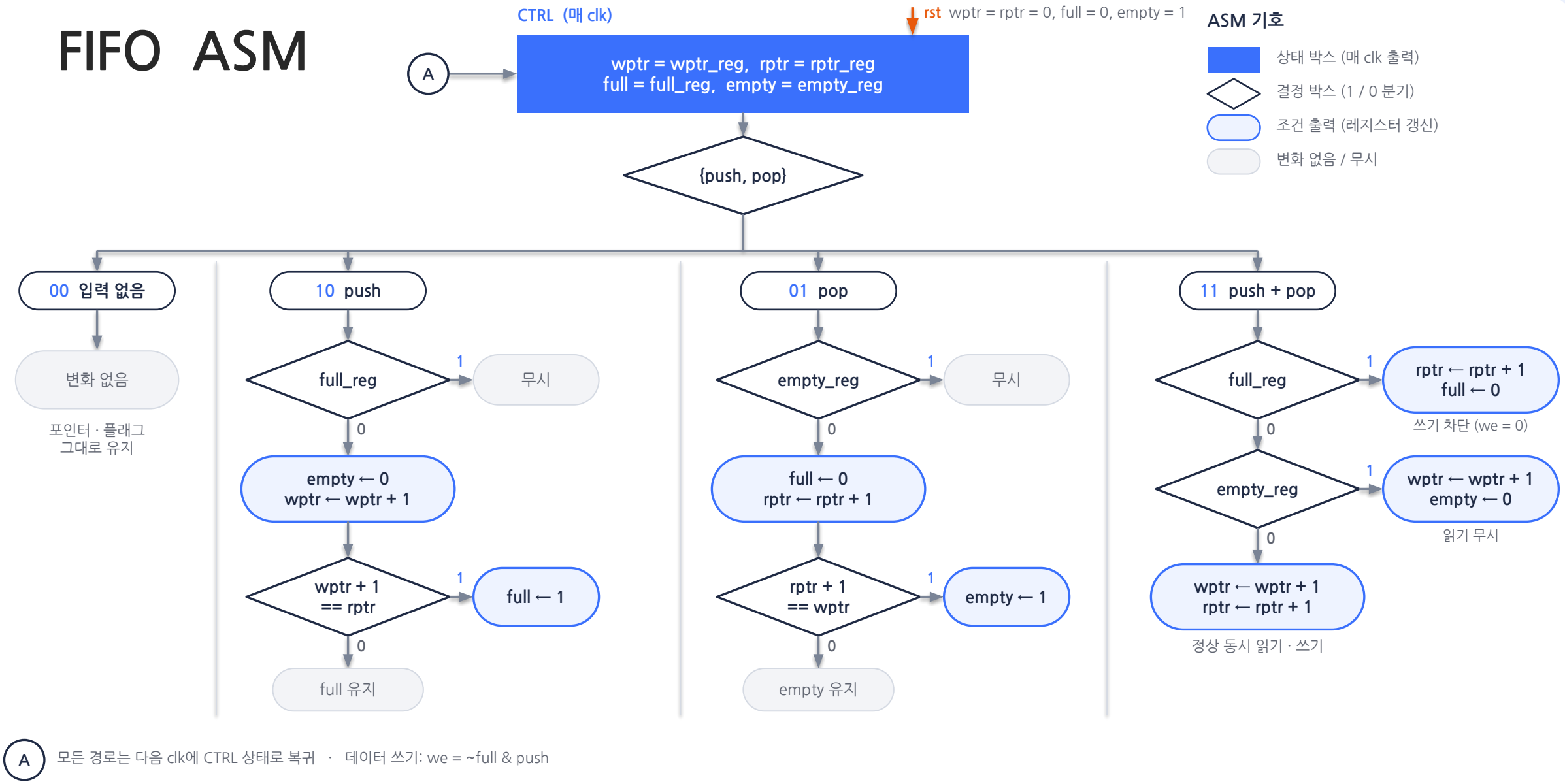
2 pop (읽기)

- pop_data는 항상 rptr 칸을 출력
- pop 시 rptr + 1, empty면 무시

3 full · empty 판정

- push 후 wptr + 1 == rptr → full
- pop 후 rptr + 1 == wptr → empty
- 포인터가 같을 때 두 상태를 플래그로 구분

참고 자료


FIFO ASM
CTRL (매 clk)
rst wptr = rptr = 0, full = 0, empty = 1
A
wptr = wptr_reg, rptr = rptr_reg
full = full_reg, empty = empty_reg
{push, pop}
ASM 기호
상태 박스 (매 clk 출력)
결정 박스 (1 / 0 분기)
조건 출력 (레지스터 갱신)
변화 없음 / 무시
00 입력 없음
변화 없음
포인터 · 플래그 그대로 유지
10 push
full_reg
1
무시
0
empty ← 0
wptr ← wptr + 1
wptr + 1 == rptr
1
full ← 1
0
full 유지
01 pop
empty_reg
1
무시
0
full ← 0
rptr ← rptr + 1
rptr + 1 == wptr
1
empty ← 1
0
empty 유지
11 push + pop
full_reg
1
rptr ← rptr + 1
full ← 0
쓰기 차단 (we = 0)
0
empty_reg
1
wptr ← wptr + 1
empty ← 0
읽기 무시
0
wptr ← wptr + 1
rptr ← rptr + 1
정상 동시 읽기 · 쓰기
A
모든 경로는 다음 clk에 CTRL 상태로 복귀 · 데이터 쓰기: we = ~full & push